

Bootstrappable Fully Homomorphic Attribute-Based Encryption with Unbounded Circuit Depth

Feixiang Zhao¹, Shixin Chen², Man Ho Au¹, Jian Weng², Huaxiong Wang³,
and Jian Guo³

¹ The Hong Kong Polytechnic University, Hong Kong, China
fxzhao37@gmail.com, mhaau@polyu.edu.hk

² Jinan University, Guangzhou, China
ceciliachan9710@gmail.com, cryptjweng@gmail.com

³ Nanyang Technological University, Singapore
hxwang@ntu.edu.sg, guojian@ntu.edu.sg

Abstract. Homomorphic attribute-based encryption (HABE) is a useful cryptographic primitive that supports both fine-grained access control and computation over ciphertexts. However, existing HABE schemes are limited to the homomorphic evaluation of circuits with either bounded depth or a restricted number of inputs. To address this problem, we introduce a bootstrappable, fully homomorphic attribute-based encryption (FHABE) scheme that supports computations of circuits with unbounded depth over cross-attribute ciphertexts. Compared to state-of-the-art schemes, the proposed scheme also has a more lightweight ciphertext and eliminates the reliance on the random oracle model. Additionally, we extend the FHABE to a proxy re-encryption setting, proposing an attribute-based homomorphic proxy re-encryption scheme that facilitates efficient sharing of encrypted data. This scheme is the first lattice-based multi-hop attribute-based proxy re-encryption scheme with unbounded hops of re-encryptions, which may be of independent interest.

Keywords: Attribute-based encryption · Homomorphic encryption · Unbounded depth · Bootstrapping · Proxy re-encryption.

1 Introduction

Attribute-based encryption (ABE) [42] is a public key cryptosystem designed to enable fine-grained access control of ciphertexts. ABE is typically categorized into two types. In ciphertext-policy ABE (CP-ABE), each decryption key sk_x is associated with an attribute set, e.g., $x \in \mathbb{Z}_q^l$, which represents a structured encoding of real-life attributes (such as roles, departments, or clearances) as vectors (over $\mathbb{Z}_q$). Each ciphertext ct_f is associated with an access policy e.g., an arithmetic function $f: \mathbb{Z}_q^l \rightarrow \mathbb{Z}_q$, which determines decryption eligibility based on the user's attribute set (e.g., if and only if $f(x) = 0$). Conversely, in key-policy ABE (KP-ABE), each ciphertext ct_x is embedded with an attribute set $x \in \mathbb{Z}_q^l$

and each decryption key sk_f is associated with an access policy f that specifies which ciphertexts can be decrypted ($f(\mathbf{x}) = 0$). In this paper, we mainly focus on the key-policy scenario. Regarding quantum resistance, numerous expressive lattice-based ABE schemes were proposed [9, 28, 6, 14].

Homomorphic encryption (HE) is a cryptographic primitive that allows computations over encrypted data without the need of decryption. In 2009, Gentry [25] first demonstrated the feasibility of fully homomorphic encryption (FHE) that supports arbitrary function evaluations by using the bootstrapping theorem. Specifically, bootstrapping refreshes a ciphertext to a lower error state by homomorphically evaluating the decryption circuit on it using an encrypted secret key, ensuring successful subsequent computations. Based on Gentry’s blueprint, a sequence of more practical FHE schemes [44, 13, 10, 22, 12, 27] were developed. A substantial breakthrough in FHE bootstrapping came with the introduction of FHEW by Ducas and Micciancio [21], which notably reduced bootstrapping times to a sub-second level. GINX/TFHE scheme [18, 19] was proposed later to improve performance by operating over the torus structure and leveraging binary secret keys. Subsequent research continued to enhance this framework, leading to a series of optimized schemes [19, 37, 45, 32], with better efficiency.

However, traditional FHE schemes are one-to-one models, where each encryption is associated with a single user, and homomorphic computations are confined to ciphertexts under the same user as well. This setting may not be ideal for multi-user scenarios. For example, consider a large organization that outsources encrypted data to a cloud server. It might be necessary for this data to be accessible and manipulable by a designated set of staff members, rather than by a single individual. While there do exist multi-user extensions of FHE such as threshold homomorphic encryption [5, 8] and multi-key HE (MKHE) [38, 40, 31] schemes that address certain multi-user requirements, these solutions often lack fine-grained access control over which users may decrypt or compute on specific data. This limitation highlights the need for a fine-grained access model in many practical applications.

Homomorphic attribute-based encryption (HABE), incorporating FHE and ABE, is a cryptosystem that simultaneously addresses the computations and the fine-grained access control on ciphertexts. Gentry et al. [27] first showed how to build a HABE using their GSW-FHE, but their scheme can only handle computations over ciphertexts under the same attribute. To enable computations of cross-attribute ciphertexts, Brakerski et al. [11] first introduced the notion of target HABE (T-HABE). Based on the number of supported policies during computations, T-HABE is classified into two categories. Single-target HABE (ST-HABE) [11] supports homomorphic computations of some ciphertexts $\{\text{ct}_{\mathbf{x}_i}\}$ whose embedded attribute sets $\{\mathbf{x}_i\}$ satisfy a single policy f , i.e., for all $\mathbf{x}_i$, $f(\mathbf{x}_i) = 0$. In contrast, multi-target HABE (MT-HABE) [11, 30] enables computations over ciphertexts $\{\text{ct}_{\mathbf{x}_i}\}$ associated with a set of policies $F = \{f_j\}$. That is, for any $\mathbf{x}_i$, there exists some $f_j \in F$ with $f_j(\mathbf{x}_i) = 0$. Current MT-HABE schemes are built upon the composition of HABE and MKHE techniques. In general, this is achieved by first transforming each ciphertext $\text{ct}_{\mathbf{x}_i}$ into an expanded ciphertext

$\text{ct}_{\mathbf{x}_i}^{(F)}$ under a composite key sk_F derived from the set of keys $\{\text{sk}_{f_j}\}$, which allows single-key homomorphic evaluation over the transformed ciphertexts.

However, these current ST-HABE and MT-HABE schemes are limited to processing circuits with a prior bounded depth, which is often insufficient for practical applications requiring more complex computations post-setup. Moreover, the parameter size in these schemes tends to increase significantly with the required circuit depth, posing serious scalability limitations. Thus, unbounded HABE has particular significance in real-world applications. Here we provide a more detailed analysis of the limitations and challenges associated with the current state-of-the-art HABE constructions.

Current Limitations and Challenges. The current state-of-the-art single-target BCTW-HABE scheme [11] involves public parameters including random matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{B}_0, \dots, \mathbf{B}_l \in \mathbb{Z}_q^{n \times N}$, and a random vector $\mathbf{v} \in \mathbb{Z}_q^n$, where $m = \Theta(n \lceil \log q \rceil)$, $N = n \lceil \log q \rceil$, and $M = (m + N + 1) \lceil \log q \rceil$. Let $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^\top$ and $\mathbf{G}_{m+N+1} = \mathbf{I}_{m+N+1} \otimes \mathbf{g}^\top$, where $\mathbf{g} = (1, 2, \dots, 2^{\lceil \log q \rceil})$. $\mathbf{G}^{-1}(\mathbf{A})$ is some decomposition algorithm such that $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{A}) = \mathbf{A}$. A fresh ciphertext for a message $\mu \in \{0, 1\}$ under an attribute set $\mathbf{x} \in \{0, 1\}^l$, consists of two matrices: $\mathbf{C} = [\mathbf{A}|\mathbf{B}_0|\mathbf{v}]^\top \cdot \mathbf{S} + \mu \mathbf{G}_{m+N+1} + \mathbf{E} \in \mathbb{Z}_q^{(m+N+1) \times M}$ and $\mathbf{C}_{\mathbf{x}} = [\mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_l + x_l \mathbf{G}]^\top \cdot \mathbf{S} + \mathbf{E}_{\mathbf{x}} \in \mathbb{Z}_q^{l \times M}$, where $\mathbf{S}$ is a random matrix, $\mathbf{E}, \mathbf{E}_{\mathbf{x}}$ are some error terms bounded by B . For a policy f , let $\mathbf{B}_f$ be the matrix corresponding to f . The secret key is a vector, $\mathbf{t}_f = [\mathbf{r}_f | \mathbf{r}'_f | 1]$, such that $\mathbf{A} \cdot \mathbf{r}_f + (\mathbf{B}_0 + \mathbf{B}_f) \cdot \mathbf{r}'_f + \mathbf{v} = \mathbf{0}$. Here, $\mathbf{r}'_f$ is a random vector in $\{0, 1\}$ generated by a random oracle, and $\mathbf{r}_f$ is sampled such that $\|\mathbf{r}_f\|_\infty \leq \tau \sqrt{m}$ for some parameter τ .

The decryption and computation operate over the functioned ciphertext as, $\hat{\mathbf{C}}_f = [\mathbf{A}|\mathbf{B}_0 + \mathbf{B}_f + f(\mathbf{x})\mathbf{G}|\mathbf{v}]^\top \cdot \mathbf{S} + \mu \mathbf{G}_{m+N+1} + \mathbf{E}_f$, which can be efficiently constructed from $\mathbf{C}$ and $\mathbf{C}_{\mathbf{x}}$. $\mathbf{E}_f$ is some error with bound $(M+1)^{d_f}$ for the policy depth bound d_f . Let the vector $\mathbf{u} = (0, \dots, 0, \lfloor q/2 \rfloor) \in \mathbb{Z}_q^{m+N+1}$. If $f(\mathbf{x}) = 0$, the decryption proceeds by

$$\mathbf{t}_f^\top \cdot \hat{\mathbf{C}}_f \mathbf{G}^{-1}(\mathbf{u}) = \mu(\mathbf{t}_f^\top \otimes \mathbf{g}^\top) \mathbf{G}^{-1}(\mathbf{u}) + \mathbf{t}_f^\top \mathbf{E}_f \mathbf{G}^{-1}(\mathbf{u}) = \lfloor q/2 \rfloor \mu + \underbrace{\mathbf{t}_f^\top \mathbf{E}_f \mathbf{G}^{-1}(\mathbf{u})}_{\text{error}}$$

To evaluate the product $\mu^{(1)} \cdot \mu^{(2)}$, BCTW-HABE computes the homomorphic multiplication: $\hat{\mathbf{C}}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)})$. It holds that:

$$\begin{aligned} \mathbf{t}_f^\top \cdot \hat{\mathbf{C}}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)}) &= \mu^{(2)} \mathbf{t}_f^\top \mathbf{G}_{m+N+1} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)}) + \mathbf{t}_f^\top \cdot \mathbf{E}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)}) \\ &= \mu^{(1)} \mu^{(2)} \mathbf{t}_f^\top \mathbf{G}_{m+N+1} + \underbrace{\mu^{(2)} \mathbf{t}_f^\top \mathbf{E}_f^{(1)} + \mathbf{t}_f^\top \cdot \mathbf{E}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)})}_{\text{error}} \end{aligned}$$

Based on the decryption, the final error term of the evaluated ciphertext is:

$$\left(\mu^{(2)} \mathbf{t}_f^\top \mathbf{E}_f^{(1)} + \mathbf{t}_f^\top \cdot \mathbf{E}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)}) \right) \cdot \mathbf{G}^{-1}(\mathbf{u})$$

The homomorphic NAND evaluation, $\text{NAND}(\mu^{(1)}, \mu^{(2)})$ can then be performed via $\text{NAND}(\hat{\mathbf{C}}_f^{(1)}, \hat{\mathbf{C}}_f^{(2)}) = \mathbf{G}_{m+N+1} - \hat{\mathbf{C}}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)})$, with the same size of error.

We observe that compared to an original ciphertext, the error grows by a factor of $O(M + 1)$ per multiplication. To evaluate a circuit with depth d_G , correct decryption requires that the accumulated error satisfies: $\|(\mu^{(2)} \mathbf{t}_f^\top \mathbf{E}_f^{(1)} + \mathbf{t}_f^\top \cdot \mathbf{E}_f^{(2)} \cdot \mathbf{G}^{-1}(\hat{\mathbf{C}}_f^{(1)}) \cdot \mathbf{G}^{-1}(\mathbf{u})\|_\infty \leq (\tau m \sqrt{m} + N + 1)B(1 + M)^{d_G} \leq q/4$, where B is the original error bound. Therefore, the maximum supported circuit depth of the BCTW-HABE scheme is bounded by:

$$d_G \leq \log_M \left(\frac{q}{4(\tau m \sqrt{m} + N + 1)B} \right)$$

which imposes a significant constraint on the computational capacity.

As shown, a key limitation of the state-of-the-art HABE, i.e., BCTW, is the inability to control the error growth in homomorphically evaluated ciphertexts. Hiromasa and Kawai [30] proposed a MT-HABE scheme that allows arbitrary additional ciphertexts input during homomorphic computation based on the BCTW framework. However, their scheme inherits the same problem of error expansion. Clear and McGoldrick [20] proposed a generic construction of HABE that supports circuits with unbounded depth. However, their work did not provide a concrete instantiation, and the proposed scheme remains an incomplete solution: it restricts the number of inputs and supports only 1-hop computations-meaning the evaluated ciphertexts cannot be further computed. Additionally, these schemes rely on the random oracle model for secret key generation. Table 1 summarizes the results of current HABE schemes.

Bootstrapping offers a potential solution to reduce the accumulated error. However, obtaining a bootstrappable FHABE by a direct extension from a current HABE is particularly challenging. This is because bootstrapping requires the encryption of secret key bits as part of the public key and necessitates non-interactively deriving attributes or policy information from such key. This raises the question: *Can we construct a bootstrappable FHABE scheme that supports unbounded circuit depth without imposing restrictions on the number of input?*

1.1 Our Contributions

To address the limitations previously identified, this paper introduces a lattice-based bootstrappable single-target fully homomorphic attribute-based encryption (ST-FHABE) scheme that supports both fine-grained access control and homomorphic computations on cross-attribute ciphertexts. Compared to prior state-of-the-art HABE schemes, our scheme offers the following key advantages: (1) support for evaluating circuits of unbounded depth, (2) getting rid of the random oracle model, and (3) more lightweight ciphertexts.

Specifically, our FHABE scheme is constructed with the following novelties: (1) Building upon the BGG-ABE framework for its native support of vector encryption, our scheme achieves a more compact form of ciphertext compared to prior HABE approaches based on bit encryption. (2) We provide a new in-depth analysis of the decryption circuit in BGG-ABE framework. (3) Through

Table 1: Comparison of the results (functionality and size) between lattice-based HABE schemes. n is the LWE dimension, q is the ciphertext modulus, l is the attribute set length, and g_w is the number of gates in access policy.

Scheme	ST/MT	HE	Model	Ciphertext Size Per Message	Secret Key
GSW [27]	Single-Attribute	Bounded (Depth)	Standard	$O(l^2 n^2 \log^5 q)$	$O(g_w \cdot n^2 \log^3 q)$
BCTW [11]	ST & MT	Bounded (Depth)	Random Oracle	$O(ln^2 \log^4 q)$	$O(n \log^2 q)$
HK [30]	MT	Bounded (Depth)	Random Oracle	$O((l + \log q) \cdot (n^3 \log^6 q))$	$O(n \log^3 q)$
CM [20]	MT	Restricted (Input)	Random Oracle	No Instance	-
Ours	ST	Unbounded (FHE)	Standard	$O(l \log q)$	$O(n^2 \log^3 q)$

this analysis, we introduce a method to embrace the FHEW bootstrapping techniques, which refreshes the evaluated ciphertexts to a low noise state. (4) Finally, to address the problems of ciphertext expansion and malformed key after bootstrapping, we introduce a new "key-switching" technique called auxiliary recovery key that conveys the access control information. This key helps to recover the bootstrapped ciphertexts back to their original correct form, enabling successful subsequent computations.

Furthermore, we extend our FHABE to a proxy re-encryption (PRE) setting, introducing a multi-hop attribute-based fully homomorphic proxy re-encryption (AB-FHPRE) scheme that additionally facilitates flexible sharing of encrypted data. Notably, this scheme is the first lattice-based multi-hop attribute-based proxy re-encryption capable of unbounded hops of re-encryptions, which may be of independent interest.

1.2 Technical Overview

In general, built upon the BGG-ABE framework and incorporating FHEW bootstrapping techniques, our proposed scheme involves a new analysis of BGG-ABE decryption circuit that the homomorphic computation works over, a new method for implementing bootstrapping techniques to control noise growth during computation of unbounded-depth circuits, and a new key-switching technique that maintains the correct form of resulting ciphertexts. Before going through these techniques, we first review the BGG-ABE and FHEW frameworks, along with some necessary related concepts.

Throughout the paper, for elements, like a_1, a_2 , we use (a_1, a_k) or $[a_1 || a_2]$ to denote the vertical concatenation, and $[a_1 | a_2]$ to represent the horizontal concatenation. For given LWE parameters n and q , we use the gadget matrix $\mathbf{G}_n = \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times n \lceil \log q \rceil}$, where $\mathbf{g} = (1, 2, \dots, 2^{\lceil \log q \rceil}) \in \mathbb{Z}_q^{\lceil \log q \rceil}$. For sim-

plicity, we sometimes omit the subscript when it is clear from context. Associated with $\mathbf{G}$, we use two key algorithms, $\mathbf{G}^{-1}$ and P2, as follows:

- $\mathbf{G}^{-1}(\mathbf{v})$: takes a vector $\mathbf{v} \in \mathbb{Z}_q^n$, outputs $\mathbf{u} = (\mathbf{u}_1, \dots, \mathbf{u}_n) \in \{0, 1\}^{n \lceil \log q \rceil}$ where $\mathbf{u}_i \in \{0, 1\}^{\lceil \log q \rceil}$ and $\mathbf{v}[i] = \sum_{j=0}^{\lceil \log q \rceil} 2^j \cdot \mathbf{u}_i[j]$.
- P2($\mathbf{X}$): takes a matrix $\mathbf{X} \in \mathbb{Z}_q^{n \times k}$, outputs the matrix $\mathbf{G}^\top \cdot \mathbf{X} \in \mathbb{Z}_q^{n \lceil \log q \rceil \times k}$.

The $\mathbf{G}^{-1}$ algorithms can be directly extended to matrix inputs by applying $\mathbf{G}^{-1}$ column-wisely to each column of the input matrix. It can also be verified that: For any $\mathbf{X} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{X}) = \mathbf{X} \in \mathbb{Z}_q^{n \times m}$; For any $\mathbf{X} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{Y} \in \mathbb{Z}_q^{m \times k}$, it holds $\text{P2}(\mathbf{X})^\top \cdot \mathbf{G}^{-1}(\mathbf{Y}) = \mathbf{X}^\top \cdot \mathbf{Y} \in \mathbb{Z}_q^{m \times k}$.

Review of BGG-ABE. We revisit the framework of BGG-ABE scheme [6] on which our FHABE is based. In a BGG-ABE scheme supporting attribute sets of length l , the attribute set space is $\mathbb{Z}_q^l$, and the family of supported policies is a set of poly-size arithmetic circuit, $\mathcal{F} = \{f : \mathbb{Z}_q^l \rightarrow \mathbb{Z}_q\}$. The public parameters consist of random chosen matrices, $\mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U} \in \mathbb{Z}_q^{n \times m}$, and the gadget matrix $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^\top$. A ciphertext ct_x corresponding to an attribute set $\mathbf{x} = (x_1, \dots, x_l) \in \mathbb{Z}_q^l$ and a message $\mu \in \{0, 1\}^m$ consists of two vectors:

$$\text{ct}_x := (\mathbf{c} = [\mathbf{A}|\mathbf{B}_1 + x_1\mathbf{G}|\dots|\mathbf{B}_l + x_l\mathbf{G}]^\top \mathbf{s} + \mathbf{e}, \mathbf{c}_{out} = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_{out} + \left\lfloor \frac{q}{t} \right\rfloor \cdot \mu)$$

where $\mathbf{s} \in \mathbb{Z}_q^n$ is a random vector, $t < q$ is the plaintext modulus, and $\mathbf{e}, \mathbf{e}_{out}$ are some error vectors. For a given policy f , there exists a public generated matrix $\mathbf{D}_{f,\mathbf{x}}$ such that $[\mathbf{B}_1 + x_1\mathbf{G}|\dots|\mathbf{B}_l + x_l\mathbf{G}] \cdot \mathbf{D}_{f,\mathbf{x}} = \mathbf{B}_f + f(\mathbf{x})\mathbf{G}$, where $\mathbf{B}_f \in \mathbb{Z}_q^{n \times m}$ is a matrix uniquely determined by f and $\mathbf{B}_1, \dots, \mathbf{B}_l$. The secret key for f is a sampled short matrix, $\mathbf{R}_f \in \mathbb{Z}_q^{2m \times m}$, such that $[\mathbf{A}|\mathbf{B}_f] \cdot \mathbf{R}_f = \mathbf{U}$. Then the decryption proceeds as:

- Let $\mathbf{c} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l)$ and $\mathbf{e} = (\mathbf{e}_{in}, \mathbf{e}_1, \dots, \mathbf{e}_l)$ such that $\mathbf{c}_{in} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_{in}$, $\mathbf{c}_i = (\mathbf{B}_i + x_i\mathbf{G})^\top \mathbf{s} + \mathbf{e}_i$. Compute $\mathbf{c}_f = \mathbf{D}_{f,\mathbf{x}}^\top \cdot [\mathbf{c}_1 || \dots || \mathbf{c}_l] = (\mathbf{B}_f + f(\mathbf{x})\mathbf{G})^\top \cdot \mathbf{s} + \mathbf{e}_f$, where $\mathbf{e}_f = \mathbf{D}_{f,\mathbf{x}}^\top \cdot [\mathbf{e}_1 || \dots || \mathbf{e}_l]$.
- Construct the functioned ciphertext $\hat{\text{ct}}_f = (\hat{\mathbf{c}}_f, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$, where $\hat{\mathbf{c}}_f = [\mathbf{c}_{in} || \mathbf{c}_f] = [\mathbf{A}|\mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \mathbf{s} + [\mathbf{e}_{in} || \mathbf{e}_f] \in \mathbb{Z}_q^{2m}$.
- Compute $\lfloor q/t \rfloor \cdot \mu \approx \mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f = \lfloor q/t \rfloor \cdot \mu + \mathbf{R}_f^\top \cdot [\mathbf{e}_{in} || \mathbf{e}_f]$.

It holds that if $f(\mathbf{x}) = 0$, the decryption will succeed once the error term holds $\|\mathbf{R}_f^\top \cdot [\mathbf{e}_{in} || \mathbf{e}_f]\|_\infty \leq q/2t$.

Review of FHEW Computation and Bootstrapping. Our FHABE leverages the FHEW bootstrapping techniques to reduce the error of evaluated ciphertexts. Here, we review the general procedure of FHEW computation and bootstrapping [21, 37]. More details are provided in Appendix A and Appendix B. Let $\text{LWE}_s^{t/q}(\mu, \beta) = (\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ be a LWE encryption of $\mu \in \{0, 1\}$, under the secret key $\mathbf{s} \in \mathbb{Z}_q^n$, ciphertext modulus q , plaintext modulus t , and the

error bound β . FHEW supports the evaluation of binary logic gates on fresh LWE ciphertexts, e.g., $\mathbf{c}^{(1)} \in \text{LWE}_s^{4/q}(\mu^{(1)}, q/16)$ and $\mathbf{c}^{(2)} \in \text{LWE}_s^{4/q}(\mu^{(2)}, q/16)$. Considering the NAND gate, the computation proceeds in two steps:

- **Simple arithmetic evaluation.** First, an intermediate ciphertext is computed by a simple arithmetic operation corresponding to gate (e.g., addition for NAND) on $\mathbf{c}^{(1)}$ and $\mathbf{c}^{(2)}$. This yields an LWE encryption $\mathbf{c}' = (\mathbf{a}^{(1)} + \mathbf{a}^{(2)}, b^{(1)} + b^{(2)}) \in \text{LWE}_s^{4/q}(\mu^{(1)} + \mu^{(2)}, q/8)$, with increased noise.
- **Bootstrapping.** Next, a bootstrapping algorithm $\text{Bootstrap}_{\text{NAND}}$ is run to $\mathbf{c}'$. This algorithm takes as input $\mathbf{c}'$, a bootstrapping key (btk), and a key switching key (ksk) — both of which are part of the public key. It outputs $\mathbf{c}'' \leftarrow \text{Bootstrap}_{(\text{NAND})}(\text{btk}, \text{ksk}, \mathbf{c}') \in \text{LWE}_s^{4/q}(\mu^{(1)} \bar{\wedge} \mu^{(2)}, q/16)$, a refreshed ciphertext that encrypts the NAND result and reduces the noise level to that of a fresh ciphertext.

The resulting ciphertext $\mathbf{c}''$ can be used in further homomorphic operations with reduced error.

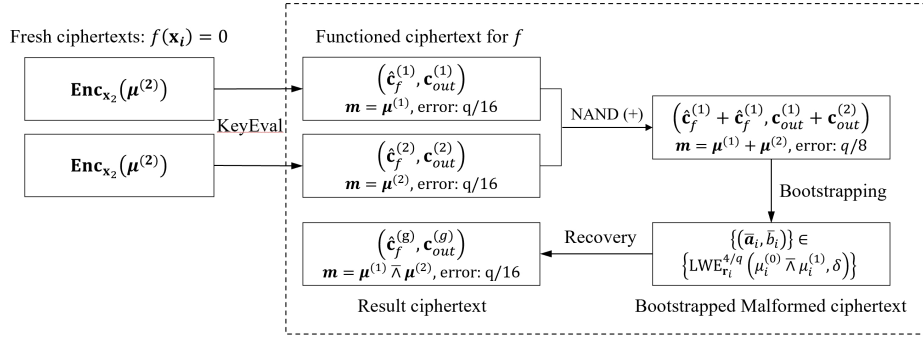


Fig. 1: Cycle for a simple homomorphic NAND gate evaluation of FHABE

Our Techniques. Our construction begins by a new observation and analysis of decryption circuit of the BGG-ABE.

New Analysis of BGG-ABE Decryption. As mentioned above, the decryption of BGG-ABE is performed by $\mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f$. We observe that the decryption process can be alternatively represented as follows. Let the underlying message $\boldsymbol{\mu} = (\mu_1, \dots, \mu_m) \in \{0, 1\}^m$, $\mathbf{c}_{out} = (b_1, \dots, b_m) \in \mathbb{Z}_q^m$, $\hat{\mathbf{c}}_f = (a_1, \dots, a_{2m}) \in \mathbb{Z}_q^{2m}$, and the secret key $\mathbf{R}_f = [\mathbf{r}_1 \dots \mathbf{r}_m] \in \mathbb{Z}_q^{2m \times m}$, where $\mathbf{r}_i \in \mathbb{Z}_q^{2m}$. The decryption process is as:

$$\mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f = \begin{bmatrix} b_1 - \langle \mathbf{r}_1, \hat{\mathbf{c}}_f \rangle \\ \vdots \\ b_m - \langle \mathbf{r}_m, \hat{\mathbf{c}}_f \rangle \end{bmatrix} \approx \lfloor q/t \rfloor \cdot \boldsymbol{\mu} = \begin{bmatrix} \lfloor q/t \rfloor \cdot \mu_1 \\ \vdots \\ \lfloor q/t \rfloor \cdot \mu_m \end{bmatrix}$$

Thus, we can observe that each pair of $(\hat{\mathbf{c}}_f, b_i) \in \mathbb{Z}_q^{2m+1}$ for $i \in [m]$ is a LWE encryption of μ_i under the secret key $\mathbf{r}_i \in \mathbb{Z}_q^{2m}$.

To accommodate the binary vector encryption inherent in the BGG-ABE's framework, we intend to employ the FHEW techniques to support Boolean circuit evaluation on attribute-based ciphertexts and to effectively manage error accumulation.

Homomorphic Computation. We consider the NAND evaluation of two fresh ciphertexts, $\text{ct}_{\mathbf{x}^{(1)}}$ that encrypts the message $\boldsymbol{\mu}^{(1)} = (\mu_1^{(1)} \cdots, \mu_m^{(1)})$ under attribute set $\mathbf{x}^{(1)}$, and $\text{ct}_{\mathbf{x}^{(2)}}$ that encrypts the message $\boldsymbol{\mu}^{(2)} = (\mu_1^{(2)} \cdots, \mu_m^{(2)})$ under attribute set $\mathbf{x}^{(2)}$, where $f(\mathbf{x}^{(1)}) = f(\mathbf{x}^{(2)}) = 0$. The computation operates over the corresponding functioned ciphertexts, $\hat{\text{ct}}_f^{(1)} = (\hat{\mathbf{c}}_f^{(1)}, \mathbf{c}_{out}^{(1)})$ and $\hat{\text{ct}}_f^{(2)} = (\hat{\mathbf{c}}_f^{(2)}, \mathbf{c}_{out}^{(2)})$. Our scheme use the plaintext modulus $t = 4$. For $j = 1, 2$, let $\hat{\mathbf{c}}_f^{(j)} = (a_1^{(j)}, \dots, a_{2m}^{(j)}) \in \mathbb{Z}_q^{2m}$ and $\mathbf{c}_{out}^{(j)} = (b_1^{(j)}, \dots, b_m^{(j)}) \in \mathbb{Z}_q^m$. Based on the above analysis of decryption circuit, we note that each $(\hat{\mathbf{c}}_f^{(j)}, b_i^{(j)})$ is a LWE encryption of $\mu_i^{(j)}$ for $i \in [m]$ and $j = 1, 2$. Our idea here is to set the appropriate parameter satisfying $(\hat{\mathbf{c}}_f^{(j)}, b_i^{(j)}) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(j)}, q/16)$, such that we can implement element-wise FHEW NAND evaluation to each pair of $(\hat{\mathbf{c}}_f^{(1)}, b_i^{(1)})$ and $(\hat{\mathbf{c}}_f^{(2)}, b_i^{(2)})$ as follows:

- Compute $\mathbf{a} = \hat{\mathbf{c}}_f^{(1)} + \hat{\mathbf{c}}_f^{(2)} = (a_1^{(1)} + a_1^{(2)}, \dots, a_{2m}^{(1)} + a_{2m}^{(2)}) \in \mathbb{Z}_q^{2m}$, $\mathbf{b} = (b_1, \dots, b_m) = \mathbf{c}_{out}^{(1)} + \mathbf{c}_{out}^{(2)} = (b_1^{(1)} + b_1^{(2)}, \dots, b_m^{(1)} + b_m^{(2)}) \in \mathbb{Z}_q^m$, such that for each $i \in [m]$, $(\mathbf{a}, b_i) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(1)} + \mu_i^{(2)}, q/8)$.
- Then for each $i \in [m]$, run $\bar{\mathbf{c}}_i = (\bar{\mathbf{a}}_i, \bar{b}_i) \leftarrow \text{Bootstrap}_{(\text{NAND})}(\text{btk}, \text{ksk}, (\mathbf{a}, b_i)) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(1)} \bar{\wedge} \mu_i^{(2)}, q/16)$, with some btk and ksk related to $\mathbf{r}_i$, such that the resulting error holds $\|\bar{e}_i\|_\infty \leq \delta$ for some bound $\delta \leq q/16$.

After the bootstrapping, the resulting message $\bar{\boldsymbol{\mu}} = (\mu_1^{(1)} \bar{\wedge} \mu_1^{(2)}, \dots, \mu_m^{(1)} \bar{\wedge} \mu_m^{(2)})$ is encrypted by a sequence of bootstrapped $(\bar{\mathbf{a}}_i, \bar{b}_i)$. Let $\bar{\mathbf{c}}_f = (\bar{\mathbf{a}}_1, \dots, \bar{\mathbf{a}}_m) \in \mathbb{Z}_q^{2mm}$, $\bar{\mathbf{c}}_{out} = (\bar{b}_1, \dots, \bar{b}_m) \in \mathbb{Z}_q^m$, and $\bar{\mathbf{e}} = (\bar{e}_1, \dots, \bar{e}_m) \in \mathbb{Z}_q^m$. Then it holds that

$$(\bar{\mathbf{c}}_f, \bar{\mathbf{c}}_{out}) \text{ is an encryption of } \bar{\boldsymbol{\mu}} \text{ under a new expanded key } \bar{\mathbf{R}}_f = \begin{bmatrix} \mathbf{r}_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{r}_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{r}_m \end{bmatrix} \in \mathbb{Z}_q^{2mm \times m}, \text{ such that:}$$

$$\bar{\mathbf{c}}_{out} - \bar{\mathbf{R}}_f^\top \cdot \bar{\mathbf{c}}_f = \begin{bmatrix} \bar{b}_1 - \langle \bar{\mathbf{a}}_1, \mathbf{r}_1 \rangle \\ \vdots \\ \bar{b}_m - \langle \bar{\mathbf{a}}_m, \mathbf{r}_m \rangle \end{bmatrix} = \begin{bmatrix} \frac{q}{4}(\mu_1^{(1)} \bar{\wedge} \mu_1^{(2)}) + \bar{e}_1 \\ \vdots \\ \frac{q}{4}(\mu_m^{(1)} \bar{\wedge} \mu_m^{(2)}) + \bar{e}_m \end{bmatrix} = \frac{q}{4} \bar{\boldsymbol{\mu}} + \bar{\mathbf{e}}$$

However, such bootstrapped ciphertext introduces two critical issues: (1) The size of the bootstrapped functioned ciphertext, i.e., $[\bar{\mathbf{c}}_f \| \bar{\mathbf{c}}_{out}] \in \mathbb{Z}_q^{2mm} \times \mathbb{Z}_q^m$, grows

by a factor $O(m)$ compared to the original one, e.g., $\hat{\mathbf{ct}}_f^{(1)} = [\hat{\mathbf{c}}_f^{(1)} \parallel \mathbf{c}_{out}^{(1)}] \in \mathbb{Z}_q^{3m}$.
 (2) The associated key, $\mathbf{R}_f \in \mathbb{Z}_q^{2mm \times m}$, differs from the original expected key, $\mathbf{R}_f \in \mathbb{Z}_q^{2m \times m}$, also with a $O(m)$ expansion. These discrepancy hinder the ability to perform continuous homomorphic operations. Thus, to achieve unbounded computations, it is necessary to develop a new method that generates bootstrapped ciphertexts in the original compact form.

Auxiliary Recovery Key. To address the mentioned problems, our approach utilizes an auxiliary recovery key, which recovers the bootstrapped ciphertext back to the original form, such that it can be properly decrypted by the original secret key $\mathbf{R}_f$. Specifically, we use a recovery key $\mathbf{ark}_f$, as part of the public key, that conveys the access control information as follows:

$$\mathbf{ark}_f = \begin{bmatrix} [\mathbf{A} \mid \mathbf{B}_f]^\top \mathbf{E}_1 + \mathbf{E}_2 & \mathbf{0}_{2m \times m} \\ \mathbf{U}^\top \mathbf{E}_1 + \mathbf{E}_3 - \mathbf{P}2(\mathbf{R}_f)^\top & \mathbf{I}_{m \times m} \end{bmatrix} \in \mathbb{Z}_q^{3m \times (2mm \lceil \log q \rceil + m)}$$

where $\mathbf{E}_1$ is a random matrix from $\mathbb{Z}_q$, $\mathbf{E}_2, \mathbf{E}_3$ are two error matrices. To keep the consistency of notation, we use $\hat{\mathbf{ct}}_f^{(g)}$ to denote the homomorphically evaluated ciphertext under the boolean circuit g and policy f (here $g = \text{NAND}$). The restoration process is processed by the computation over $\mathbf{ark}_f$ and the bootstrapped ciphertext, $(\bar{\mathbf{c}}_f, \bar{\mathbf{c}}_{out})$ as follows:

$$\hat{\mathbf{ct}}_f^{(g)} = \begin{bmatrix} \hat{\mathbf{c}}_f^{(g)} \\ \mathbf{c}_{out}^{(g)} \end{bmatrix} = \mathbf{ark}_f \cdot \begin{bmatrix} \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) \\ \bar{\mathbf{c}}_{out} \end{bmatrix} = \begin{bmatrix} [\mathbf{A} \mid \mathbf{B}_f]^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}} \\ \mathbf{U}^\top \tilde{\mathbf{s}} + \lfloor q/4 \rfloor \tilde{\boldsymbol{\mu}} + \tilde{\mathbf{e}}_{out} \end{bmatrix} \in \mathbb{Z}_q^{3m}$$

where $\tilde{\mathbf{s}} = \mathbf{E}_1 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)$ functioned as some new random vector, and $\tilde{\mathbf{e}} = \mathbf{E}_2 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)$, and $\tilde{\mathbf{e}}_{out} = \mathbf{E}_3 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) + \tilde{\mathbf{e}}$ and some error terms. As a result, the final ciphertext $\hat{\mathbf{ct}}_f^{(g)} = (\hat{\mathbf{c}}_f^{(g)}, \mathbf{c}_{out}^{(g)})$ is now a ciphertext with reduced error and correct form for the subsequent computations.

Note that the bootstrapping for each pair of LWE ciphertexts, $(\hat{\mathbf{c}}_f^{(1)}, b_i^{(1)})$, and $(\hat{\mathbf{c}}_f^{(2)}, b_i^{(2)})$ for $\mu_i^{(1)} \bar{\wedge} \mu_i^{(2)}$, are mutually independent. Thus, these bootstrapping operations can ideally be executed in a parallel mode, reducing the overall computation time. We further highlight that, due to the binary vector encryption inherent in the BGG-ABE framework, e.g., $\mathbf{ct}_x := (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out}) \in \mathbb{Z}_q^{(l+1)m}$ for $\boldsymbol{\mu} \in \{0, 1\}^m$, our scheme achieves a more lightweight ciphertext structure, requiring only $O(l)$ space per message. In contrast, existing HABE schemes such as BCTW-HABE support only one-bit encryption per ciphertext. Moreover, our construction is proven secure in the standard model, while prior works rely on the random oracle model to generate part of the secret keys.

Selection of FHE. Our scheme employs FHEW framework to enable bootstrapping of evaluated ciphertexts. This choice is motivated by FHEW's native support for Boolean circuit computation, which aligns with the binary encryption structure inherent in mainstream lattice-based ABE [6, 14, 33]. In contrast,

frameworks such as BGV [12, 24], BFV [22, 29, 39] and CKKS [17, 16, 7, 3] are better suited for arithmetic circuit evaluations. While TFHE framework [18, 19, 45] supports Boolean circuits with slightly better efficiency, it imposes a binary key space constraint, which is incompatible with the key space over $\mathbb{Z}_q$ required by the ABE sampling algorithm (see Section 2.6).

Extension to Multi-Hop AB-FHPRE. We also show how to extend our FHABE construction to a multi-hop AB-FHPRE scheme capable of an unbounded number of re-encryptions. Built upon the FHABE, this scheme additionally allows a proxy to convert a ciphertext for a policy f_1 to a ciphertext for another policy f_2 , without altering the underlying plaintext, realizing efficient encrypted data sharing. The core idea of this extension is the design of a re-encryption key as:

$$\text{rek}_{f_1 \rightarrow f_2} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_{f_2}]^\top \mathbf{E}_1^{(r)} + \mathbf{E}_2^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{U}^\top \mathbf{E}_1^{(r)} + \mathbf{E}_3^{(r)} - \text{P2}(\mathbf{R}_{f_1})^\top & \mathbf{I}_{m \times m} \end{bmatrix}$$

However, the re-encrypted ciphertext (e.g. $\text{ct}_{f_2}^{(r)}$) generated using this re-encryption key has uncontrollable error accumulation, making further re-encryption infeasible. To accommodate the unbounded hops of re-encryptions, our method incorporates the use of a homomorphic AND evaluation of $\bar{\text{ct}}_{f_2}^{(r)}$ and itself, i.e., $\text{AND}(\bar{\text{ct}}_{f_2}^{(r)}, \bar{\text{ct}}_{f_2}^{(r)})$, with a bootstrapping, $\text{Bootstrap}_{(\text{AND})}$. The key here is the AND evaluation of a binary message and itself yields the same message. Thus, this operation acts as a refreshing process to reduce the error of the re-encrypted ciphertext for next re-encryption.

2 Preliminaries

2.1 Notation

Throughout this paper, we use $[n]$ to denote the set $\{1, 2, \dots, n\}$. We use $x \leftarrow X$ to denote the process of sampling a value x over the distribution X , and $x \xleftarrow{\$} X$ to denote the uniformly random sampling process. We use a bold lower-case letter like $\mathbf{a}$ to denote a column vector and a bold upper-case letter like $\mathbf{A}$ to denote a matrix.

Regarding the norms representation, we define $\|\mathbf{a}\|$ as the l_2 norm of the vector $\mathbf{a} = (a_1, \dots, a_k)$ and $\|\mathbf{a}\|_\infty$ as $\|a_i\|_{\max}$. For a matrix $\mathbf{A} \in \mathbb{Z}^{k \times m}$, let $\tilde{\mathbf{A}}$ be the Gram-Schmidt (GS) orthogonalization of $\mathbf{A}$. We use $\|\mathbf{A}\|$ to denote the l_2 norm of the longest column of $\mathbf{A}$. We define $\|\mathbf{A}\|_{\text{GS}} = \|\tilde{\mathbf{A}}\|$ and $\|\mathbf{A}\|_2 = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$. It holds: $\|\mathbf{A}\|_{\text{GS}} \leq \|\mathbf{A}\| \leq \|\mathbf{A}\|_2 \leq \sqrt{k}\|\mathbf{A}\|$ and $\|\mathbf{A} \cdot \mathbf{S}\|_2 \leq \|\mathbf{A}\|_2 \cdot \|\mathbf{S}\|_2$.

2.2 Cyclotomic Rings

$\Phi_N(X) = \prod_{i \in \mathbb{Z}_N^*} (X - \omega_N^i)$ is called the N -th cyclotomic polynomial where ω_N is the complex N th root of unity. $\Phi_N(X)$ is a monic polynomial of degree

$\varphi(N) = |\mathbb{Z}_N^*|$. Throughout this paper, we use the $2N$ -th cyclotomic polynomial $\Phi_{2N}(X) = X^N + 1$ and its associated cyclotomic ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, where $N = 2^k$ for some integer k . Each element in $\mathcal{R}$ can be represent as a polynomial of degree $N - 1$ over $\mathbb{Z}$, i.e. $a = a_0 + a_1X + \dots, a_{N-1}X^{N-1} \in \mathcal{R}$ for $a_i \in \mathbb{Z}$. We also use $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$ to identify the residue ring of $\mathcal{R}$ modulo the integer q .

2.3 Lattices

In this paper we use full-rank lattices defined as follows: For positive integers n, m, q , a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, and a vector $\mathbf{u} \in \mathbb{Z}_q^n$, we define the following lattices: $\Lambda_q^\perp(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = 0 \pmod{q}\}$ and $\Lambda_q^{\mathbf{u}}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{u} \pmod{q}\}$. We note the fact that if $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{m \times m}$ is a basis of $\Lambda_q^\perp(\mathbf{A})$, it is a basis of $\Lambda_q^\perp(z\mathbf{A})$ for a non-zero integer $z \in \mathbb{Z}_q$.

Learning with Errors (LWE). In this section we introduce the decisional LWE (DLWE) problem [41] on which our proposed scheme is based.

Definition 1 (DLWE) For integers n, m, q and an error distribution χ over $\mathbb{Z}$, the $\text{DLWE}_{n,m,q,\chi}$ problem is to distinguish the following two distributions:

$$(\mathbf{A}, \mathbf{A}^\top \mathbf{s} + \mathbf{e}) \quad \text{and} \quad (\mathbf{A}, \mathbf{u})$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \chi^m, \mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^m$ are sampled independently. The DLWE assumption is that the DLWE problem is infeasible.

2.4 LWE Symmetric Encryption

In this section we recall the basic LWE symmetric encryption scheme [41, 4] on which the FHEW is based. A LWE symmetric encryption is parametrized by a dimension n , a ciphertext modulus q , and a plaintext modulus t . For a secret key $\mathbf{s} \in \mathbb{Z}_q^n$, the encryption of message $m \in \mathbb{Z}_t$ is defined as:

$$\text{LWE}_{\mathbf{s}}^{t/q}(m) = \left(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + \frac{q}{t}m + e \right) \in \mathbb{Z}_q^{n+1}$$

where $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ is chosen at uniformly random and $e \leftarrow \chi$ for which χ is an error distribution that holds the DLWE hardness assumption. The decryption of a ciphertext, $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}}^{t/q}(m)$, works as follows:

$$\left\lfloor \frac{t}{q}(b - \langle \mathbf{a}, \mathbf{s} \rangle) \right\rfloor = \left\lfloor \frac{t}{q}(\frac{q}{t}m + e) \right\rfloor = \left\lfloor m + \frac{t}{q}e \right\rfloor = m \pmod{t}$$

Note that if $|(t/q)e| < 1/2$, the message can be correctly recovered. $\text{LWE}_{\mathbf{s}}^{t/q}(m, \beta)$ denotes the ciphertext $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}}^{t/q}(m)$ with error bounded by β : $|e| < \beta$, and we drop the superscript t/q when it is obvious in the context.

RLWE Symmetric Encryption. FHEW uses the RLWE encryption [35] in the bootstrapping procedure. For a dimension $N = 2^k$, a ciphertext modulus $Q \in \mathbb{Z}$, and a plaintext modulus $t < Q$, let $z \in \mathcal{R}_Q$ be a secret, a RLWE encryption of message $\tilde{m} \in \mathcal{R}_Q$ (an encoding of m) is defined as:

$$\text{RLWE}_z^{t/Q}(\tilde{m}) = (a, a \cdot z + e + \tilde{m}) \in \mathcal{R}_Q^2$$

where $a \xleftarrow{\$} \mathcal{R}_Q$ is chosen at uniformly random, and each coefficient of e is sampled from the error distribution $\bar{\chi}$ that holds the RLWE hardness assumption. The decryption of a ciphertext, $(a, b) \in \text{RLWE}_z^{t/Q}(m)$, proceeds as:

$$b - a \cdot z = \tilde{m} + e$$

Typically, the encoding is computed by $\tilde{m} = (q/t)m$ for some plaintext modulus t and $q < Q$. The final recover of the message is computed by a decoding function:

$$m = \left\lfloor \frac{t}{q}(\tilde{m} + e) \right\rfloor.$$

RGSW Encryption. FHEW bootstrapping uses a more flexible Gadget vector: $\mathbf{g}_b = (1, B_g, \dots, B_g^{d_g-1})$, with some base B_g and $d_g = \lceil \log_{B_g} q \rceil$. For any $r \in \mathcal{R}_Q$, we define $\mathbf{g}_b^{-1}(r) = (r_0, \dots, r_{d_g-1})$, for which $\|r_i\|_\infty < B_g$ and $r = \sum_{i=0}^{d_g-1} B_g^i \cdot r_i$. We adapt the definitions of RLWE' , RGSW encryptions from [37]:

- $\text{RLWE}'_z(m) = (\text{RLWE}_z(m), \text{RLWE}_z(B_g m), \dots, \text{RLWE}_z(B_g^{d_g-1} m)) \in \mathcal{R}_Q^{2d_g}$.
- $\text{RGSW}_z(m) = (\text{RLWE}'_z(-z \cdot m), \text{RLWE}'_z(m)) \in \mathcal{R}_Q^{2d_g \times 2}$.

2.5 Gaussian Distributions

We use $\mathcal{D}_\sigma(\Lambda_q^{\mathbf{u}}(\mathbf{A}))$ defined by Regev [41] to denote a discrete Gaussian distribution on $\Lambda_q^{\mathbf{u}}(\mathbf{A})$ with the parameter $\sigma > 0$. For a matrix $\mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_k) \in \mathbb{Z}_q^{n \times k}$, we let $\Lambda_q^{\mathbf{U}}(\mathbf{A})$, as defined in [6], be a distribution over matrices in $\mathbb{Z}^{m \times k}$, where the i -th column of some $\mathbf{R} \leftarrow \Lambda_q^{\mathbf{U}}(\mathbf{A})$ is sampled from $\Lambda_q^{\mathbf{u}_i}(\mathbf{A})$. It can be checked that $\mathbf{A}\mathbf{R} = \mathbf{U} \pmod{q}$. We use the following Lemma [6]:

Lemma 1. *For integers n, m, q, d , a parameter $\sigma > 0$, and matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{U} \in \mathbb{Z}_q^{n \times k}$, if $\mathbf{R} \leftarrow \mathcal{D}_\sigma(\Lambda_q^{\mathbf{U}}(\mathbf{A}))$ and $\mathbf{S} \leftarrow \{1, -1\}^{m \times m}$, it holds that: $\|\mathbf{R}\|_2 \leq \sigma\sqrt{mk}$, $\|\mathbf{R}^\top\|_2 \leq \sigma\sqrt{mk}$, and $\|\mathbf{S}\|_2 \leq 20\sqrt{m}$ with overwhelming probability.*

Sub-Gaussian. A variable X is called Sub-Gaussian with parameter α if it holds that $E[e^{2\pi t X}] \leq e^{\pi \alpha^2 t^2}$ for all $t \geq 0$. If X is sub-Gaussian with parameter α , it holds that $\Pr[|X| \geq t] \leq 2e^{-\pi t^2 / \alpha^2}$. A variable X is called B -bounded if its supports is in $[-B, B]$, and such variable is sub-Gaussian with parameter $B\sqrt{2\pi}$.

2.6 Trapdoors Generation and Sampling Algorithms

Trapdoor Generation Algorithms. We use the following efficient algorithms to generate short trapdoor basis of lattices. For integers n, m, q :

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \mathbf{T}_\mathbf{A})$ ([2, 36]): A randomized algorithm that, given $m = \Theta(n \log q)$, outputs a full-rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor basis $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A})$, satisfying $\mathbf{A}$ is statistically close to uniform in $\mathbb{Z}_q$ and $\|\mathbf{T}_\mathbf{A}\|_{\text{GS}} = O(\sqrt{n \log q})$ with overwhelming probability.
- $\text{ExtendRight}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{B}) \rightarrow \mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$ ([15]): A deterministic algorithm that takes full-rank matrices $\mathbf{A}, \mathbf{B} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor basis $\mathbf{T}_\mathbf{A}$ of $\Lambda_q^\perp(\mathbf{A})$, outputs a trapdoor basis $\mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$ of $\Lambda_q^\perp(\mathbf{A}|\mathbf{B})$, where $\|\mathbf{T}_\mathbf{A}\|_{\text{GS}} = \|\mathbf{T}_{[\mathbf{A}|\mathbf{B}]}\|_{\text{GS}}$.
- $\text{ExtendLeft}(\mathbf{A}, \mathbf{G}, \mathbf{T}_\mathbf{G}, \mathbf{R}) \rightarrow \mathbf{T}_{[\mathbf{A}|\mathbf{G}+\mathbf{A}\mathbf{R}]}$ ([1]): A deterministic algorithm that takes a full-rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a Gadget matrix $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$, and a trapdoor basis $\mathbf{T}_\mathbf{G}$ of $\Lambda_q^\perp(\mathbf{G})$, outputs a trapdoor basis $\mathbf{T}_{[\mathbf{A}|\mathbf{G}+\mathbf{A}\mathbf{R}]}$ of $\Lambda_q^\perp(\mathbf{A}|\mathbf{G} + \mathbf{A}\mathbf{R})$, where $\|\mathbf{T}_{[\mathbf{A}|\mathbf{G}+\mathbf{A}\mathbf{R}]}\|_{\text{GS}} = \|\mathbf{T}_\mathbf{G}\|_{\text{GS}} \cdot (1 + \|\mathbf{R}\|_2)$. Note the fact that there is a public known basis $\mathbf{T}_\mathbf{G}$ of $\Lambda_q^\perp(\mathbf{G})$ with $\|\mathbf{T}_\mathbf{G}\| \leq \sqrt{5}$.

Sampling Algorithm. Here we recall an efficient algorithm for finding a short $\mathbf{R} \in \mathbb{Z}^{m \times k}$ given $\mathbf{A}$ and $\mathbf{U}$, such that $\mathbf{A}\mathbf{R} = \mathbf{U}$.

- $\text{SampleD}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{U}, \alpha) \rightarrow \mathbf{R}$ ([26]): A randomized algorithm that takes a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a trapdoor basis $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A})$, a matrix $\mathbf{U} \in \mathbb{Z}_q^{n \times k}$, and a parameter $\alpha = \|\mathbf{T}_\mathbf{A}\|_{\text{GS}} \cdot \omega(\sqrt{\log m})$, outputs a randomly sampled $\mathbf{R} \in \mathbb{Z}_q^{m \times k}$ which is statistically close to $\mathcal{D}_\alpha(\Lambda_q^\mathbf{U}(\mathbf{A}))$ satisfying $\mathbf{A}\mathbf{R} = \mathbf{U}$.

Lemma 2 (Leftover Hash Lemma). *Suppose that $m > (n+1) \log q + \omega(\log n)$. Let $\mathbf{S} \xleftarrow{\$} \{1, -1\}^{m \times k}$ where k is polynomial in n . Let $\mathbf{A}$ and $\mathbf{U}$ are matrices chosen uniformly in $\mathbb{Z}_q^{n \times m}$ and $\mathbb{Z}_q^{n \times k}$ respectively. Then for all vector $\mathbf{e} \in \mathbb{Z}_q^m$, the distribution $(\mathbf{A}, \mathbf{A}\mathbf{S}, \mathbf{S}^\top \mathbf{e})$ is statistically close to the distribution $(\mathbf{A}, \mathbf{U}, \mathbf{S}^\top \mathbf{e})$.*

2.7 Key-Homomorphic Evaluation Algorithms

In this section we summarize four efficient deterministic key-homomorphic evaluation algorithms, $\text{KeyEval}_{\text{pk}}$, $\text{KeyEval}_{\text{D}}$, $\text{KeyEval}_{\text{ct}}$, and $\text{KeyEval}_{\text{sim}}$. These algorithms hold the following properties for the family of policy circuits $\mathcal{F} = \{f : \mathbb{Z}_q^l \rightarrow \mathbb{Z}_q\}$ with a maximum depth $d_{\mathcal{F}}$:

- $\text{KeyEval}_{\text{pk}}(f, \{\mathbf{B}_1, \dots, \mathbf{B}_l\}) \rightarrow \mathbf{B}_f$: takes $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m}$ and $f \in \mathcal{F}$, outputs $\mathbf{B}_f \in \mathbb{Z}_q^{n \times m}$ which is associated with f .
- $\text{KeyEval}_{\text{D}}(f, \mathbf{x}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\}) \rightarrow \mathbf{D}_{f, \mathbf{x}}$: takes a policy $f \in \mathcal{F}$, an attribute set $\mathbf{x} \in \mathbb{Z}_q^l$, and $\{\mathbf{B}_1, \dots, \mathbf{B}_l\}$, outputs a matrix $\mathbf{D}_{f, \mathbf{x}} \in \mathbb{Z}_q^{l \times m}$ such that $[\mathbf{B}_1 + x_1 \mathbf{G}] \cdots [\mathbf{B}_l + x_l \mathbf{G}] \cdot \mathbf{D}_{f, \mathbf{x}} = \mathbf{B}_f + f(\mathbf{x}) \mathbf{G}$ and $\|\mathbf{D}_{f, \mathbf{x}}\|_\infty \leq (m+1)^{d_{\mathcal{F}}}$.
- $\text{KeyEval}_{\text{ct}}(f, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i=1}^l) \rightarrow \mathbf{c}_f$: takes $f \in \mathcal{F}$, $\mathbf{x} \in \mathbb{Z}_q^l$, $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{c}_i = (\mathbf{B}_i + x_i \mathbf{G})^\top \mathbf{s} + \mathbf{e}_i$ for some $\mathbf{e}_i$ with bound B , evaluates $\mathbf{D}_{f, \mathbf{x}} \leftarrow \text{KeyEval}_{\text{D}}(f, \mathbf{x}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$, outputs $\mathbf{c}_f = \mathbf{D}_{f, \mathbf{x}}^\top \cdot [\mathbf{c}_1 \| \dots \| \mathbf{c}_l] \in \mathbb{Z}_q^m$, such that $\mathbf{c}_f = [\mathbf{B}_f + f(\mathbf{x}) \mathbf{G}]^\top \cdot \mathbf{s} + \mathbf{e}_f$ and $\|\mathbf{e}_f\|_\infty \leq (m+1)^{d_{\mathcal{F}}} \cdot B$.

- $\text{KeyEval}_{\text{sim}}(f, \mathbf{x}^*, \{\mathbf{S}_i\}_{i=1}^l, \mathbf{A}) \rightarrow \mathbf{S}_f$: on input $f \in \mathcal{F}$, $\mathbf{x}^* = (x_1^*, \dots, x_t^*) \in \mathbb{Z}_q^l$, $\mathbf{S}_i \leftarrow \{1, -1\}^{m \times m}$ and $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, outputs $\mathbf{S}_f \in \mathbb{Z}_q^{m \times m}$, satisfying $\mathbf{A}\mathbf{S}_f - f(\mathbf{x}^*)\mathbf{G} = \mathbf{B}_f$, such that $\mathbf{B}_f \leftarrow \text{KeyEval}_{\text{pk}}(f, \{\mathbf{A}\mathbf{S}_i - x_i^*\mathbf{G}\}_{i=1}^l)$ and $\|\mathbf{S}_f\|_2 \leq (m+1)^{d_{\mathcal{F}}}$ with overwhelming probability.

2.8 FHEW Framework for Bootstrapping

In this section, we introduce the FHEW bootstrapping framework that our FHABE scheme works on. As the full details of FHEW are intricate, we focus here on the key algorithms and components relevant to our construction, and defer more details to Appendix B. The FHEW scheme enables homomorphic evaluation of Boolean circuits over LWE ciphertexts, under the same secret key. It leverages a bootstrapping algorithm to reduce the noise of evaluated ciphertexts, guaranteeing that the output remains correct for continued computation.

Key Switching. FHEW bootstrapping includes a procedure known as key switching, which converts a LWE encryption under a key $\mathbf{z} \in \mathbb{Z}_q^N$ to a LWE encryption under a different key $\mathbf{s} \in \mathbb{Z}_q^n$, while preserving the underlying plaintext. This process is realized by a public precomputed key switch key, $\mathbf{ksk}$, defined as:

$$\mathbf{ksk} = \left\{ \mathbf{k}_{h,i,j} = \text{LWE}_{\mathbf{s}}^{q/q}(hz_i B_{\mathbf{ks}}^j) \right\}_{h \in [0, B_{\mathbf{ks}} - 1], i \in [N], j \in [0, d_{\mathbf{ks}} - 1]}$$

where $B_{\mathbf{ks}}$ is a decomposition base chosen for key switching and $d_{\mathbf{ks}} = \lceil \log_{B_{\mathbf{ks}}} q \rceil$.

Bootstrapping key. The bootstrapping algorithm also relies on a public precomputed bootstrapping key, which is encryption for bits of the secret key $\mathbf{s} \in \mathbb{Z}_q^n$:

$$\mathbf{btk} = \left\{ \text{RGSW}_z(X^{v \cdot s_i \cdot B_r^j}) \right\}_{v \in [0, B_r - 1], i \in [n], j \in [0, d_r - 1]}$$

where $z \leftarrow \mathcal{R}_Q$ is a chosen RLWE secret, B_r is a decomposition base chosen for bootstrapping, and $d_r = \lceil \log_{B_r} q \rceil$.

Considering NAND computation on two LWE ciphertexts, $\mathbf{c}^{(1)} = (\mathbf{a}^{(1)}, b^{(1)}) \in \text{LWE}_{\mathbf{s}}^{4/q}(\mu^{(1)}, q/16)$, $\mathbf{c}^{(2)} = (\mathbf{a}^{(2)}, b^{(2)}) \in \text{LWE}_{\mathbf{s}}^{4/q}(\mu^{(2)}, q/16)$. We first compute:

$$\mathbf{c}' = (\mathbf{a}^{(1)} + \mathbf{a}^{(2)}, b^{(1)} + b^{(2)}) \in \text{LWE}_{\mathbf{s}}^{4/q}(\mu^{(1)} + \mu^{(2)}, q/8) \quad (1)$$

There exists the following bootstrapping algorithm for a precomputed bootstrapping key, $\mathbf{btk} = \{\text{RGSW}_z(X^{v \cdot s_i \cdot B_r^j})\}_{v,i,j}$, that is a sequence of RGSW encryptions with degree $N = q/2$, a secret $z \leftarrow \mathcal{R}_Q$, and an error distribution $\bar{\chi}$ of parameter $\bar{\sigma}$, and a precomputed key switch key, $\mathbf{ksk} = \{\text{LWE}_{\mathbf{s}}^{q/q}(hz_i' B_{\mathbf{ks}}^j)\}_{h,i,j}$, as a sequence of LWE encryptions for $\mathbf{z}' = (z_0, -z_{q/2-1}, \dots, -z_1)$, where $\mathbf{z}$ is the coefficient vector of z , under the secret key $\mathbf{s}$ and an error distribution χ of parameter σ :

- $\text{Bootstrap}_{(\text{NAND})}(\mathbf{ksk}, \mathbf{btk}, \mathbf{c}') \rightarrow \mathbf{c}''$: takes as input $\mathbf{c}' \in \text{LWE}_{\mathbf{s}}^{4/q}(\bar{\mu}, q/8)$, $\mathbf{btk}$, and $\mathbf{ksk}$ as above, outputs $\mathbf{c}'' \in \text{LWE}_{\mathbf{s}}^{4/q}(\bar{\mu}, \delta)$.

Fact 1 *Assuming the hardness of $\text{RLWE}_{\mathcal{R}, Q, \bar{\chi}}$ and based on the central limit heuristic, there exists an algorithm $\text{Bootstrap}_{(\text{NAND})}$, such that the error of its output has a Gaussian distribution with parameter δ' :*

$$\delta' = \sqrt{\frac{q^2}{Q^2}(\bar{\sigma}^2 \frac{B_r^2}{12} d_{rq} N(B_g + \frac{Q}{B_g^{d_g}} - 1) + \sigma^2 N d_{ks}) + \frac{\|\mathbf{s}\|^2 + 1}{12}} \quad (2)$$

and the error has the bound $\delta = \delta' / \sqrt{2\pi}$.

Leveraging the above fact, in a practical scheme, we choose the appropriate parameters to ensure that the resulting ciphertext remains within a small enough noise bound δ , enabling correct subsequent computations.

2.9 Fully Homomorphic Attribute-Based Encryption

Definition 2 (FHABE) *A fully homomorphic attribute-based encryption scheme $\Pi = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ consists of the following algorithms:*

- $\text{Setup}(1^\lambda, 1^l) \rightarrow (\text{pp}, \text{msk})$: On input a security parameter λ and the length of attribute vectors, l , output the public parameters pp and a master secret key msk .
- $\text{KeyGen}(\text{pp}, \text{msk}, f) \rightarrow (\text{pk}_f, \text{sk}_f)$: On input public parameters pp and an access policy circuit f , output a public key pk_f and a decryption secret key sk_f .
- $\text{Enc}(\text{pp}, \mathbf{x}, \mu) \rightarrow \text{ct}_{\mathbf{x}}$: On input an attribute vector $\mathbf{x} \in \{0, 1\}^l$ and a message $\mu \in \mathcal{M}$, output a ciphertext $\text{ct}_{\mathbf{x}}$ along with the attribute $\mathbf{x}$.
- $\text{Dec}(\text{pp}, \text{sk}_f, \text{ct}_{\mathbf{x}}) \rightarrow \mu$: On input a secret key sk_f of policy f and a ciphertext $\text{ct}_{\mathbf{x}}$ under an attribute $\mathbf{x}$, output the error symbol $\perp$ if $f(\mathbf{x}) \neq 0$; Otherwise, output a message μ .
- $\text{Eval}(\text{pp}, \text{pk}_f, (\text{ct}_{\mathbf{x}^{(1)}}, \dots, \text{ct}_{\mathbf{x}^{(k)}}), g) \rightarrow \text{ct}_f^{(g)}$: On input a public key pk_f of policy f , a set of ciphertext $\text{ct}_{\mathbf{x}^{(1)}} \dots, \text{ct}_{\mathbf{x}^{(k)}}$, where $\text{ct}_{\mathbf{x}^{(i)}} \in \text{Enc}(\text{pp}, \mathbf{x}^{(i)}, \mu^{(i)})$ and $\mu^{(i)} = (\mu_1^{(i)}, \dots, \mu_m^{(i)}) \in \{0, 1\}^m$ for some m , and an evaluated circuit $g \in \{0, 1\}^k \rightarrow \{0, 1\}$, output $\perp$ if the condition, $f(\mathbf{x}^{(i)}) = 0$ for all $i \in [k]$, is not satisfied; Otherwise, output an evaluated ciphertext $\text{ct}_f^{(g)}$ that is an encryption of $g(\mu^{(1)}, \dots, \mu^{(k)}) = (g(\mu_1^{(1)}, \dots, \mu_1^{(k)}), \dots, g(\mu_m^{(1)}, \dots, \mu_m^{(k)})) \in \{0, 1\}^m$ under f .

Definition 3 (Correctness) *Let $\mathcal{F}$ be a family of supported policy circuits, $\mathcal{G} = \{g : \{0, 1\}^* \rightarrow \{0, 1\}\}$ be a family of evaluation circuits, and $\mathcal{X}$ be an attribute space. We say a FHABE scheme $\Pi = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ is correct if for all $\lambda \in \mathbb{N}$, $l \in \mathbb{N}$, $f \in \mathcal{F}$, $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^l)$, and $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, the following holds:*

- For a message $\mu \in \{0, 1\}^*$, $\mathbf{x} \in \mathcal{X}^l$, and a ciphertext $\text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}, \mu)$, it holds that if $f(\mathbf{x}) = 0$:

$$\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \text{ct}_{\mathbf{x}}) = \mu] \geq 1 - \text{negl}(\lambda)$$

- For k messages $\mu^{(1)}, \dots, \mu^{(k)} \in \{0, 1\}^*$, attribute vectors $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)} \in \mathcal{X}^l$, ciphertexts $\text{ct}_{\mathbf{x}^{(1)}}, \dots, \text{ct}_{\mathbf{x}^{(k)}}$ where $\text{ct}_{\mathbf{x}^{(i)}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^{(i)}, \mu^{(i)})$, an evaluated circuit $g \in \mathcal{G}$, and an evaluated ciphertext for circuit g , $\hat{\text{ct}}_f^{(g)} \leftarrow \text{Eval}(\text{pp}, \text{pk}_f, \{\text{ct}_{\mathbf{x}^{(1)}}, \dots, \text{ct}_{\mathbf{x}^{(k)}}\}, g)$, it holds if $f(\mathbf{x}^{(i)}) = 0$ for all $i \in [k]$:

$$\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \hat{\text{ct}}_f^{(g)}) = g(\mu^{(1)}, \dots, \mu^{(k)})] \geq 1 - \text{negl}(\lambda)$$

Security for FHABE. The security definition of a FHABE scheme is similar to the framework of the state-of-the-art KP-ABE in [6], except that now we need to answer the public key pk_f in key queries. Let Π be a FHABE as described above. We consider the following game, $\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(\lambda)$, between an adversary $\mathcal{A}$ and the challenger.

1. **Initial.** $\mathcal{A}$ outputs a target attribute vector $\mathbf{x}^* \in \mathcal{X}^l$ of its own choice.
2. **Setup.** The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^l)$ and sends pp to $\mathcal{A}$.
3. **Query Phase 1.** $\mathcal{A}$ can issue polynomial times of key queries to the oracle:
 - $\mathcal{O}^{\text{KeyGen}}(f)$: On input a policy f , run $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, if $f(\mathbf{x}^*) = 0$, output pk_f ; Otherwise, output $(\text{pk}_f, \text{sk}_f)$.
4. **Challenge.** $\mathcal{A}$ chooses a pair of messages, μ_0, μ_1 , on its own choice and sends it to the challenger. The challenger randomly chooses a bit $\beta \in \{0, 1\}$, computes $\text{ct}_\beta^* \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \mu_\beta)$, sends ct_β^* to $\mathcal{A}$ as the challenge ciphertext.
5. **Query Phase 2.** $\mathcal{A}$ can continuously issue queries as in Query Phase 1.
6. **Guess.** $\mathcal{A}$ outputs a bit β' . The game outputs 1 if $\beta' = \beta$; Otherwise, the game outputs 0.

The game above is a selective security game under the chosen plaintext attack (CPA), and the advantage of the adversary in this game is defined as:

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(\lambda) := |\Pr[\beta' = \beta] - 1/2|$$

Definition 4 (Security for FHABE) *An FHABE scheme is selective IND-CPA secure if for any PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(\lambda) \leq \text{negl}(\lambda)$.*

3 Unbounded FHABE Scheme from Lattices

In this section, we present the concrete construction and correctness analysis of our proposed lattice-based bootstrappable unbounded FHABE scheme.

3.1 Construction

For a family of policy circuits $\mathcal{F} = \{f : \mathbb{Z}_q^l \rightarrow \mathbb{Z}_q\}$ with a depth bound $d_{\mathcal{F}}$, and a security parameter λ , our construction is parametrized by: $n = \text{poly}(\lambda)$, $q = 2^d$. Let $m = \Theta(n \log q)$, $\alpha_{\mathcal{F}} = (m + 1)^{d_{\mathcal{F}}}$, $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ be a Gadget matrix, and χ be a B_χ -bounded error distribution so that the $\text{DLWE}_{n, q, \chi}$ problem is hard. Set $\sigma = \omega(\alpha_{\mathcal{F}} \cdot \sqrt{\log m})$ where $\alpha_{\mathcal{F}} > \sqrt{n \log m}$ for the correctness of

the trapdoor algorithms. For the RLWE encryption in bootstrapping, set the RLWE modulus $Q = B_g^{d_g}$ for some base B_g , and $N = q/2$ as the dimension. Choose a bootstrapping base B_r and a key switching base B_{ks} . Let $\bar{\chi}$ be an error distribution of parameter $\bar{\sigma}$ such that the $\text{RLWE}_{N,Q,\bar{\chi}}$ problem is hard. Let $\text{pp}^{\text{LWE}} = (n, m, q, \chi, \sigma)$ and $\text{pp}^{\text{RLWE}} = (N, Q, B_g, B_r, B_{ks}, \bar{\chi}, \bar{\sigma})$.

For the message space $\mathcal{M} = \{0, 1\}^m$, a FHABE scheme includes the algorithms:

- **Setup**($1^\lambda, 1^l$) $\rightarrow$ (**pp**, **msk**): Choose the parameters as described above, do:
 1. Generate $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m}$ is a basis of $\Lambda_q^\perp(\mathbf{A})$.
 2. Choose $l + 1$ random matrices $\mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$.
 3. Output the public parameter $\text{pp} = (\text{pp}^{\text{LWE}}, \text{pp}^{\text{RLWE}}, \mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U}, \mathbf{G})$ and the master secret key $\text{msk} = \mathbf{T}_{\mathbf{A}}$.
- **KeyGen**(**pp**, **msk**, f) $\rightarrow$ (**pk_f**, **sk_f**): On input the public parameters **pp**, the master secret key **msk**, and an access policy circuit f , do as follows:
 1. Evaluate $\mathbf{B}_f \leftarrow \text{KeyEval}_{\text{pk}}(f, \mathbf{B}_1, \dots, \mathbf{B}_l)$.
 2. Compute $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]} \leftarrow \text{ExtendRight}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{B}_f)$.
 3. Sample $\mathbf{R}_f \leftarrow \text{SampleD}(\mathbf{A}|\mathbf{B}_f, \mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]}, \mathbf{U}, \sigma)$, such that $[\mathbf{A}|\mathbf{B}_f]\mathbf{R}_f = \mathbf{U}$.
 4. Let $\mathbf{R}_f = [\mathbf{r}_1 | \dots | \mathbf{r}_m] \in \mathbb{Z}_q^{2m \times m}$ where $\mathbf{r}_i \in \mathbb{Z}_q^{2m}$ for $i \in [m]$, compute $\text{btk}_f = \{\text{btk}_{\mathbf{r}_k}\}_{k=1}^m$ where $\text{btk}_{\mathbf{r}_k} = \{\mathbf{Z}_{h,i,j}^{(k)} = \text{RGSW}_z(X^{h \cdot \mathbf{r}_k[i] \cdot B_r^j})\}$ for some secret $z \leftarrow \mathcal{R}_Q, h \in [0, B_r - 1], i \in [2m], j \in [0, d_r - 1], d_r = \lceil \log_{B_r} q \rceil$.
 5. Compute key switch key $\text{ksk}_f = \{\text{ksk}_{\mathbf{r}_k}\}_{k=1}^m$ where $\text{ksk}_{\mathbf{r}_k} = \{\mathbf{k}_{h,i,j}^{(k)} = \text{LWE}_{\mathbf{r}_k}^{q/q}(hz_i' B_{ks}^j)\}$ for $\mathbf{z}' = (z_0, -z_{N-1}, \dots, -z_1), h \in [0, B_{ks} - 1], i \in [N], j \in [0, d_{ks} - 1]$, and $d_{ks} = \lceil \log_{B_{ks}} q \rceil$.
 6. Construct $\bar{\mathbf{R}}_f = \begin{bmatrix} \mathbf{r}_1 & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{r}_2 & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{r}_m \end{bmatrix} \in \mathbb{Z}_q^{2mm \times m}$. Then select any attribute vector $\mathbf{x}' = (x'_1, \dots, x'_l) \in \mathbb{Z}_q^l$ such that $f(\mathbf{x}') = 0$. Construct $\bar{\mathbf{M}}_1 = [\mathbf{A}|\mathbf{B}_1 + x'_1 \mathbf{G} | \dots | \mathbf{B}_l + x'_l \mathbf{G}]^\top \mathbf{E}_1 + \bar{\mathbf{E}}_2$ and $\mathbf{M}_2 = \mathbf{U}^\top \mathbf{E}_1 + \mathbf{E}_3 - \text{P2}(\bar{\mathbf{R}}_f)^\top$, where $\mathbf{E}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times 2mmd}$, $\bar{\mathbf{E}}_2 \leftarrow \chi^{(l+1)m \times 2mmd}$, and $\mathbf{E}_3 \leftarrow \chi^{m \times 2mmd}$.
 7. Evaluate $\mathbf{D}_{f,\mathbf{x}'} \leftarrow \text{KeyEval}_{\text{D}}(f, \mathbf{x}', \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$. Then compute the auxiliary recovery key $\text{ark}_f = \begin{bmatrix} \mathbf{M}_1 & \mathbf{0}_{2m \times m} \\ \mathbf{M}_2 & \mathbf{I}_{m \times m} \end{bmatrix} \in \mathbb{Z}_q^{3m \times (2mmd+m)}$, where $\mathbf{M}_1 = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f,\mathbf{x}'} \end{bmatrix}^\top \cdot \bar{\mathbf{M}}_1 = [\mathbf{A}|\mathbf{B}_f]^\top \mathbf{E}_1 + \mathbf{E}_2$, for some $\|\mathbf{E}_2\|_\infty < \alpha_{\mathcal{F}} \cdot B_\chi$.
 8. Output the secret key $\text{sk}_f = \mathbf{R}_f$ and public key $\text{pk}_f = (\text{btk}_f, \text{ksk}_f, \text{ark}_f)$.
- **Enc**(**pp**, $\mathbf{x}, \boldsymbol{\mu}$) $\rightarrow$ **ct_x**: On input an attribute vector $\mathbf{x} \in \mathbb{Z}_q^l$ and a message $\boldsymbol{\mu} \in \{0, 1\}^m$, do as follows:
 1. Choose a vector $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ at uniformly random.
 2. Choose l matrices $\mathbf{S}_i \xleftarrow{\$} \{1, -1\}^{m \times m}$ for $i \in [l]$ at uniformly random.

3. Sample errors $\mathbf{e}_0, \mathbf{e}_{out} \leftarrow \chi^m$ and compute $\mathbf{e} = [\mathbf{I}_m | \mathbf{S}_1 | \cdots | \mathbf{S}_l]^\top \cdot \mathbf{e}_0 = (\mathbf{e}_{in}, \mathbf{e}_1, \dots, \mathbf{e}_l) \in \mathbb{Z}_q^{(l+1)m}$.
 4. Construct $\mathbf{H}_\mathbf{x} = [\mathbf{A} | \mathbf{B}_1 + x_1 \mathbf{G} | \cdots | \mathbf{B}_l + x_l \mathbf{G}] \in \mathbb{Z}_q^{n \times (l+1)m}$.
 5. Compute $\mathbf{c} = \mathbf{H}_\mathbf{x}^\top \mathbf{s} + \mathbf{e} \in \mathbb{Z}_q^{(l+1)m}$ and $\mathbf{c}_{out} = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_{out} + \lfloor q/4 \rfloor \cdot \boldsymbol{\mu} \in \mathbb{Z}_q^m$.
Let $\mathbf{c} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l)$ for $\mathbf{c}_{in} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_{in}$, $\mathbf{c}_i = (\mathbf{B}_i + x_i \mathbf{G})^\top \mathbf{s} + \mathbf{e}_i$.
 6. Output the ciphertext $\mathbf{ct}_\mathbf{x} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out}) \in \mathbb{Z}_q^{(l+2)m}$ along with the attribute vector $\mathbf{x}$.
- $\text{Dec}(\text{pp}, \text{sk}_f, \text{ct}) \rightarrow \boldsymbol{\mu}$: On input a secret key $\text{sk}_f = \mathbf{R}_f$ and a ciphertext ct , do as follows based on different cases:
 1. If ct is a fresh ciphertext, let ct be $\mathbf{ct}_\mathbf{x} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$, output $\perp$ if $f(\mathbf{x}) \neq 0$; Otherwise, evaluate $\text{KeyEval}_{\text{ct}}(f, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i=1}^l) \rightarrow \mathbf{c}_f = [\mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \cdot \mathbf{s} + \mathbf{e}_f$, where $\mathbf{e}_f = \mathbf{D}_{f, \mathbf{x}}^\top \cdot [\mathbf{e}_1 \| \cdots \| \mathbf{e}_l]$. Then construct the functioned ciphertext $\hat{\mathbf{c}}_f = (\hat{\mathbf{c}}_f, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$, where $\hat{\mathbf{c}}_f = [\mathbf{c}_{in} \| \mathbf{c}_f] = [\mathbf{A} | \mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \mathbf{s} + [\mathbf{e}_{in} \| \mathbf{e}_f] \in \mathbb{Z}_q^{2m}$.
 2. If ct is an evaluated ciphertext under f , let ct be $\hat{\mathbf{c}}_f^{(g)} = (\hat{\mathbf{c}}_f, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$.
 3. Output $\boldsymbol{\mu} = \left\lfloor (4/q)(\mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f) \right\rfloor \pmod{4}$, where $\boldsymbol{\mu} \in \{0, 1\}^m$.

For the homomorphic evaluation algorithm Eval , we describe the NAND gate computation Eval.NAND here.

- $\text{Eval.NAND}(\text{pp}, \text{pk}_f, \text{ct}^{(1)}, \text{ct}^{(2)}) \rightarrow \hat{\mathbf{c}}_f^{(g)}$: On input a pk_f and two fresh or evaluated ciphertexts, $\text{ct}^{(1)}$ that is an encryption of $\boldsymbol{\mu}^{(1)} = (\mu_1^{(1)} \cdots, \mu_m^{(1)})$, and $\text{ct}^{(2)}$ that is an encryption of $\boldsymbol{\mu}^{(2)} = (\mu_1^{(2)} \cdots, \mu_m^{(2)})$, do:
 1. For each $i \in \{1, 2\}$ if $\text{ct}^{(i)}$ is a fresh ciphertext, let it be $\mathbf{ct}_{\mathbf{x}^{(i)}} = (\mathbf{c}_{in}^{(i)}, \mathbf{c}_1^{(i)}, \dots, \mathbf{c}_l^{(i)}, \mathbf{c}_{out}^{(i)})$, which satisfies $f(\mathbf{x}^{(i)}) = 0$, evaluate $\mathbf{c}_f^{(i)} \leftarrow \text{KeyEval}_{\text{ct}}(f, \mathbf{x}^{(i)}, \{\mathbf{B}_j, \mathbf{c}_j^{(i)}\}_{j=1}^l)$, construct $\hat{\mathbf{c}}_f^{(i)} = [\mathbf{c}_{in}^{(i)} \| \mathbf{c}_f^{(i)}] \in \mathbb{Z}_q^{2m}$; If $\text{ct}^{(i)}$ is an evaluated ciphertext under f , let $\text{ct}^{(i)} = (\hat{\mathbf{c}}_f^{(i)}, \mathbf{c}_{out}^{(i)})$. Then construct the functioned ciphertext $\hat{\mathbf{c}}_f^{(i)} = (\hat{\mathbf{c}}_f^{(i)}, \mathbf{c}_{out}^{(i)}) \in \mathbb{Z}_q^{3m}$ for $i \in \{1, 2\}$.
 2. Now we have $\hat{\mathbf{c}}_f^{(1)} = (\hat{\mathbf{c}}_f^{(1)}, \mathbf{c}_{out}^{(1)})$ and $\hat{\mathbf{c}}_f^{(2)} = (\hat{\mathbf{c}}_f^{(2)}, \mathbf{c}_{out}^{(2)})$. Compute:
$$\mathbf{a} = \hat{\mathbf{c}}_f^{(1)} + \hat{\mathbf{c}}_f^{(2)} \in \mathbb{Z}_q^{2m}, \quad \mathbf{b} = (b_1, \dots, b_m) = \mathbf{c}_{out}^{(1)} + \mathbf{c}_{out}^{(2)} \in \mathbb{Z}_q^m$$
 3. For each $i \in [m]$, implement the bootstrapping:
$$\bar{\mathbf{c}}_i = (\bar{\mathbf{a}}_i, \bar{b}_i) \leftarrow \text{Bootstrap}_{(\text{NAND})}(\text{btk}_{\mathbf{r}_i}, \text{ksk}_{\mathbf{r}_i}, (\mathbf{a}, b_i)) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(1)} \wedge \mu_i^{(2)}, \delta)$$

holding $\bar{\mathbf{a}}_i \in \mathbb{Z}_q^{2m}$ and $\bar{b}_i \in \mathbb{Z}_q$ with error bound δ (from the Fact 1).

4. Construct $\bar{\mathbf{c}}_f = [\bar{\mathbf{a}}_1 \| \cdots \| \bar{\mathbf{a}}_m] \in \mathbb{Z}_q^{2mm}$ and $\bar{\mathbf{c}}_{out} = (\bar{b}_1, \dots, \bar{b}_m) \in \mathbb{Z}_q^m$.
5. Compute $\hat{\mathbf{c}}_f^{(g)} = \text{ark}_f \cdot \begin{bmatrix} \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) \\ \bar{\mathbf{c}}_{out} \end{bmatrix}$ and output $\hat{\mathbf{c}}_f^{(g)} = (\hat{\mathbf{c}}_f^{(g)}, \mathbf{c}_{out}^{(g)}) \in \mathbb{Z}_q^{3m}$.

3.2 Correctness

Here we demonstrate the correctness of the proposed unbounded FHABE scheme. We consider the decryptions of both original and evaluated ciphertexts.

1. The decryption of a fresh ciphertext, $\text{ct}_{\mathbf{x}} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$ for $f(\mathbf{x}) = 0$, operates on the functioned ciphertext, $\hat{\text{ct}}_f = (\hat{\mathbf{c}}_f, \mathbf{c}_{out})$, where $\hat{\mathbf{c}}_f = [\mathbf{c}_{in} \parallel \mathbf{c}_f] = [\mathbf{A} \parallel \mathbf{B}_f]^\top \mathbf{s} + [\mathbf{e}_{in} \parallel \mathbf{e}_f]$ and $\|[\mathbf{e}_{in} \parallel \mathbf{e}_f]\| < (\alpha_{\mathcal{F}} + 1)B_\chi$, by:

$$\begin{aligned} \mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f &= (\mathbf{U}^\top \mathbf{s} + \mathbf{e}_{out} + \lfloor \frac{q}{4} \rfloor \cdot \boldsymbol{\mu}) - \mathbf{R}_f^\top ([\mathbf{A} \parallel \mathbf{B}_f]^\top \mathbf{s} + [\mathbf{e}_{in} \parallel \mathbf{e}_f]) \\ &= \lfloor \frac{q}{4} \rfloor \cdot \boldsymbol{\mu} + (\mathbf{e}_{out} - \mathbf{R}_f^\top [\mathbf{e}_{in} \parallel \mathbf{e}_f]). \end{aligned}$$

Note that the secret key $\mathbf{R}_f$ sampled by `SampleD` holds $\|\mathbf{R}_f^\top\|_2 < 2m\sigma$ with overwhelming probability. Thus, the decryption for a fresh ciphertext succeeds if the error term satisfies $\|\mathbf{e}_{out} - \mathbf{R}_f^\top [\mathbf{e}_{in} \parallel \mathbf{e}_f]\| < q/8$ where $\|\mathbf{e}_{out} - \mathbf{R}_f^\top [\mathbf{e}_{in} \parallel \mathbf{e}_f]\| < B_\chi + 2m\sigma(\alpha_{\mathcal{F}} + 1)B_\chi$ with overwhelming probability. Hence, the correctness requires that chosen parameters satisfy:

$$B_\chi + 2m\sigma(\alpha_{\mathcal{F}} + 1)B_\chi < \frac{q}{8} \quad (3)$$

2. For a homomorphically evaluated ciphertext, we consider a ciphertext output by NAND gate evaluation of two ciphertexts $\text{ct}^{(1)}$ and $\text{ct}^{(2)}$ as:

$$\hat{\text{ct}}_f^{(g)} = (\hat{\mathbf{c}}_f^{(g)}, \mathbf{c}_{out}^{(g)}) \leftarrow \text{Eval.NAND}(\text{pp}, \text{pk}_f, \text{ct}^{(1)}, \text{ct}^{(2)})$$

The computation operates on the functioned ciphertext, $\hat{\text{ct}}_f^{(i)} = (\hat{\mathbf{c}}_f^{(i)}, \mathbf{c}_{out}^{(i)})$, where $\hat{\mathbf{c}}_f^{(i)} = [\mathbf{c}_{in}^{(i)} \parallel \mathbf{c}_f^{(i)}]$. To facilitate the correct bootstrapping, it requires the error term, $\mathbf{c}_{out}^{(i)} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f^{(i)}$, of the ciphertext to be computed, is bounded by $q/16$ as in (1). Hence:

- If $\text{ct}^{(i)}$ is a fresh ciphertext, the correctness requires the chosen parameters, with overwhelming probability, satisfies:

$$\|\mathbf{e}_{out}^{(i)} - \mathbf{R}_f^\top [\mathbf{e}_{in}^{(i)} \parallel \mathbf{e}_f^{(i)}]\| < B_\chi + 2m\sigma(\alpha_{\mathcal{F}} + 1)B_\chi < \frac{q}{16} \quad (4)$$

- If $\text{ct}^{(i)}$ is an evaluated ciphertext, we inductively assume that it holds $\mathbf{c}_{out}^{(i)} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f^{(i)} = \lfloor q/4 \rfloor \cdot \boldsymbol{\mu}^{(i)} + \mathbf{e}'^{(i)}$ for some error satisfying $\|\mathbf{e}'^{(i)}\| < q/16$, so that the bootstrapping can be correctly implemented. Then the NAND computation will proceed to compute

$$\mathbf{a} = \hat{\mathbf{c}}_f^{(1)} + \hat{\mathbf{c}}_f^{(2)} \in \mathbb{Z}_q^{2m}, \quad \mathbf{b} = (b_1, \dots, b_m) = \mathbf{c}_{out}^{(1)} + \mathbf{c}_{out}^{(2)} \in \mathbb{Z}_q^m$$

followed by a bootstrapping execution as:

$$\bar{\mathbf{c}}_i = (\bar{\mathbf{a}}_i, \bar{b}_i) \leftarrow \text{Bootstrap}(\text{btk}_{\mathbf{r}_i}, \text{ksk}_{\mathbf{r}_i}, (\mathbf{a}, b_i)) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(1)} \bar{\wedge} \mu_i^{(2)}, \delta)$$

for some error $\bar{e}_i$ with bound δ introduced in Fact 1. Let $\bar{\mathbf{c}}_f = (\bar{\mathbf{a}}_1, \dots, \bar{\mathbf{a}}_m)$, $\bar{\mathbf{c}}_{out} = (\bar{b}_1, \dots, \bar{b}_m)$, we have:

$$\bar{\mathbf{c}}_{out} - \bar{\mathbf{R}}_f^\top \cdot \bar{\mathbf{c}}_f = \begin{bmatrix} \bar{b}_1 - \langle \bar{\mathbf{a}}_1, \mathbf{r}_1 \rangle \\ \vdots \\ \bar{b}_m - \langle \bar{\mathbf{a}}_m, \mathbf{r}_m \rangle \end{bmatrix} = \begin{bmatrix} \frac{q}{4}(\mu_1^{(1)} \bar{\wedge} \mu_1^{(2)}) + \bar{e}_1 \\ \vdots \\ \frac{q}{4}(\mu_m^{(1)} \bar{\wedge} \mu_m^{(2)}) + \bar{e}_m \end{bmatrix} = \frac{q}{4}(\boldsymbol{\mu}_0 \bar{\wedge} \boldsymbol{\mu}_1) + \bar{\mathbf{e}}$$

where $\|\bar{\mathbf{e}}\|_\infty < \delta$. Then the form recovery process runs as:

$$\hat{\mathbf{c}}_f^{(g)} = \begin{bmatrix} \hat{\mathbf{c}}_f^{(g)} \\ \mathbf{c}_{out}^{(g)} \end{bmatrix} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_f]^\top \mathbf{E}_1 + \mathbf{E}_2 & \mathbf{0}_{2m \times m} \\ \mathbf{U}^\top \mathbf{E}_1 + \mathbf{E}_3 - \mathbf{P}2(\bar{\mathbf{R}}_f)^\top & \mathbf{I}_{m \times m} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) \\ \bar{\mathbf{c}}_{out} \end{bmatrix}$$

Let $\tilde{\mathbf{s}} = \mathbf{E}_1 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)$, $\tilde{\mathbf{e}} = \mathbf{E}_2 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)$, and $\tilde{\mathbf{e}}_{out} = \mathbf{E}_3 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) + \bar{\mathbf{e}}$. Thus, we have the ciphertext $\hat{\mathbf{c}}_f^{(g)} = ([\mathbf{A}|\mathbf{B}_f]^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}, \mathbf{U}^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}_{out} + \lfloor q/4 \rfloor \cdot \boldsymbol{\mu})$, and the decryption proceeds as:

$$\begin{aligned} \mathbf{c}_{out}^{(g)} - \mathbf{R}_f^\top \cdot ([\mathbf{A}|\mathbf{B}_f]^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}) &= (\mathbf{U}^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}_{out} + \lfloor \frac{q}{4} \rfloor \cdot \boldsymbol{\mu}) - (\mathbf{U}^\top \tilde{\mathbf{s}} + \mathbf{R}_f^\top \tilde{\mathbf{e}}) \\ &= \lfloor \frac{q}{4} \rfloor \cdot \boldsymbol{\mu} + (\tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top \tilde{\mathbf{e}}) \end{aligned}$$

Correct decryption requires that the error term satisfies: $\|\tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top \tilde{\mathbf{e}}\| < q/8$. Also, this error term is required to have a bound of $q/16$, such that it is correct as the input to the next computation as in (1). Therefore, the chosen parameters must satisfy:

$$\begin{aligned} \|\tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top \tilde{\mathbf{e}}\| &< \|\mathbf{E}_3 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)\| + \|\bar{\mathbf{e}}\| + \|\mathbf{R}_f^\top \cdot \mathbf{E}_2 \cdot \mathbf{G}^{-1}(\bar{\mathbf{c}}_f)\| \\ &< 2m^2 d \cdot B_\chi + \delta + 2m\sigma \cdot 2m^2 d \cdot \alpha_{\mathcal{F}} B_\chi < \frac{q}{16} \end{aligned} \quad (5)$$

$$\text{where } \delta = \sqrt{\frac{q^2}{2\pi Q^2} (\bar{\sigma}^2 \frac{B_r^2}{6} d_r q m N (B_g + \frac{Q}{B_g^{d_g}} - 1) + \sigma^2 N d_{ks}) + \frac{\|\mathbf{r}_k\|^2 + 1}{24\pi}}.$$

The general inductive idea is as follows: if condition (5) holds and the input ciphertexts used in the computation have errors with a correct bound, then the error of the resulting ciphertext will also remain within the required bound for the next round of computation. Thus, it suffices to ensure that any functioned ciphertext being used for the first time, i.e., derived directly from a fresh ciphertext, starts with error bounded by $q/16$. This initial correctness is guaranteed by condition (4).

In summary, the correctness of the proposed unbounded FHABE scheme holds if the chosen parameters satisfy the conditions (3), (4), and (5). Note that these requirements also hold for the other gates such as AND, OR, and XOR.

4 Security Proof

Theorem 1 (Security). *The FHABE scheme given in Section 3.1 is selective IND-CPA secure for classes of function $\mathcal{F}, \mathcal{G}$ under the assumptions of the circular security, the hardness of $\text{DLWE}_{n,q,\chi}$, and the hardness of $\text{RLWE}_{N,Q,\bar{\chi}}$.*

Proof. The security proof proceeds in a sequence of games:

Game 0. This is the original IND-CPA security game from Definition 4.

Game 1. In Game 1, we change the way in which the matrices, $\mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l$ in pp are generated. Let $\mathbf{x}^* = (x_0^*, \dots, x_l^*) \in \mathbb{Z}_q^l$ be the target attribute vector. $\mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l$ in Game 1 are generated as follows:

1. Choose $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ at uniformly random.
2. Choose l uniformly random matrices as $\mathbf{S}_i^* \xleftarrow{\$} \{1, -1\}^{m \times m}$ for $i \in [l]$.
3. Compute $\mathbf{B}_i = \mathbf{A}\mathbf{S}_i^* - x_i^* \mathbf{G}$ for $i \in [l]$.

In Query Phase 1 and Query Phase 2, the challenger will adaptively answer the key queries issued by the adversary $\mathcal{A}$ as follows:

- $\mathcal{O}^{\text{KeyGen}}$: On input a policy f , do as follows:
 1. Evaluate $\mathbf{B}_f \leftarrow \text{KeyEval}_{\text{pk}}(f, \{\mathbf{B}_i\}_{i=1}^l)$, $\mathbf{S}_f^* \leftarrow \text{KeyEval}_{\text{sim}}(f, \mathbf{x}^*, \{\mathbf{S}_i^*\}_{i=1}^l, \mathbf{A})$, such that $\mathbf{A}\mathbf{S}_f^* - f(\mathbf{x}^*)\mathbf{G} = \mathbf{B}_f$ and $\|\mathbf{S}_f^*\| \leq \alpha_{\mathcal{F}}$.
 2. If $f(\mathbf{x}^*) \neq 0$, do as follows:
 - Compute a trapdoor $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]} \leftarrow \text{ExtendLeft}(\mathbf{A}, f(\mathbf{x}^*)\mathbf{G}, \mathbf{T}_{\mathbf{G}}, \mathbf{S}_f^*)$.
 - Sample $\mathbf{R}_f \leftarrow \text{SampleD}([\mathbf{A}|\mathbf{B}_f], \mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]}, \mathbf{U}, \sigma)$.
 - Generate the btk_f , ksk_f , and ark_f for f as KeyGen in Section 3.1.
 - Output $\text{sk}_f = \mathbf{R}_f$ and $\text{pk}_f = (\text{btk}_f, \text{ksk}_f, \text{ark}_f)$.
 3. If $f(\mathbf{x}^*) = 0$ do as follows:
 - Compute the simulated bootstrapping key $\text{btk}_f^* = \{\text{btk}_{\mathbf{r}_k}^*\}_{k=1}^m$ where $\text{btk}_{\mathbf{r}_k}^* = \{\mathbf{Z}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathcal{R}_{Q^{2d_g \times 2}}\}$.
 - Compute the simulated key switch key $\text{ksk}_f^* = \{\text{ksk}_{\mathbf{r}_k}^*\}_{k=1}^m$ where $\text{ksk}_{\mathbf{r}_k}^* = \{\mathbf{k}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathbb{Z}_q^{n+1}\}$.
 - Construct $\bar{\mathbf{M}}_1^* = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{X}_1 \in \mathbb{Z}_q^{(l+1)m \times 2mmd}$, where $\mathbf{X}_1 \xleftarrow{\$} \mathbb{Z}_q^{m \times 2mmd}$ and $\mathbf{M}_2^* \xleftarrow{\$} \mathbb{Z}_q^{m \times 2mmd}$ are uniformly chosen.
 - Evaluate $\mathbf{D}_{f,\mathbf{x}^*} \leftarrow \text{KeyEval}_D(f, \mathbf{x}^*, \{\mathbf{B}_i\}_{i=1}^l)$, then compute $\mathbf{M}_1^* = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f,\mathbf{x}^*} \end{bmatrix}^\top \cdot \bar{\mathbf{M}}_1^* \in \mathbb{Z}_q^{2m \times 2mmd}$, and compute the simulated auxiliary recovery key: $\text{ark}_f^* = \begin{bmatrix} \mathbf{M}_1^* & \mathbf{0}_{2m \times m} \\ \mathbf{M}_2^* & \mathbf{I}_{m \times m} \end{bmatrix} \in \mathbb{Z}_q^{3m \times (2mmd+m)}$
 - Output $\text{pk}_f = (\text{btk}_f^*, \text{ksk}_f^*, \text{ark}_f^*)$.

The remainder of the game remains unchanged. We show that Game 0 is computationally indistinguishable from Game 1. Based on the property of trapdoor algorithms (Section 2.6), the matrix $\mathbf{A}$ generated by TrapGen as in Game 0 is

statistically close to uniform. Also note that matrices $\{\mathbf{S}_i^*\}_{i \in [l]}$ are used in the construction of $\{\mathbf{B}_i\}_{i \in [l]}$ in **pp** and the error term $\mathbf{e} = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{e}_0$ in the challenge ciphertext. According to Lemma 2, the distributions $(\mathbf{A}, \mathbf{AS}_i^*, \mathbf{e})$ and $(\mathbf{A}, \mathbf{V}, \mathbf{e})$, where $\mathbf{V}$ is uniformly random in $\mathbb{Z}_q^{n \times m}$, are statistically close. Thus, the matrix $\mathbf{B}_i = \mathbf{AS}_i^* - x_i^* \mathbf{G}$ is statistically close to uniformly random. Moreover, under the assumptions of RLWE hardness, LWE hardness, and circular security, the distribution of $\text{RGSW}_z(X^{h \cdot \mathbf{r}_k[i] \cdot B_i^j})$ is computationally indistinguishable from that of $\mathbf{Z}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathcal{R}_Q^{2d_g \times 2}$, and the distribution of $\text{LWE}_s^{q/q}(hz'_i B_{ks}^j)$ is computationally indistinguishable from that of $\mathbf{k}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathbb{Z}_q^{n+1}$. Furthermore, for a query of $f(\mathbf{x}^*) = 0$, we note that

$$[\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot (\mathbf{A}^\top \hat{\mathbf{E}}_1 + \hat{\mathbf{E}}_2) \quad \text{and} \quad \mathbf{U}^\top \hat{\mathbf{E}}_1 + \hat{\mathbf{E}}_3 - \text{P2}(\bar{\mathbf{R}}_f)^\top$$

for some $\hat{\mathbf{E}}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times 2mmd}$, $\hat{\mathbf{E}}_2 \leftarrow \chi^{m \times 2mmd}$, and $\hat{\mathbf{E}}_3 \leftarrow \chi^{m \times 2mmd}$, function as valid components of ark_f , from the analysis of the **Reduction from DLWE** below. Also, the following two distributions are computationally indistinguishable based on the DLWE assumption:

$$([\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top (\mathbf{A}^\top \hat{\mathbf{E}}_1 + \hat{\mathbf{E}}_2), \mathbf{U}^\top \hat{\mathbf{E}}_1 + \hat{\mathbf{E}}_3 - \text{P2}(\bar{\mathbf{R}}_f)^\top), \text{ and } (\bar{\mathbf{M}}_1^*, \mathbf{M}_2^*)$$

Thus, the auxiliary recovery key ark_f^* is a valid simulation. In summary, **Game 0** is computationally indistinguishable from **Game 1**.

Game 2. Game 2 differs from Game 1 in the way by which the challenge ciphertext, ct_β^* , are generated. In Game 2, $\text{ct}_\beta^* = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$ is chosen in $\mathbb{Z}_q^{(l+2)m}$ at uniformly random. Since now the challenge ciphertext is completely independent from β , the advantage of $\mathcal{A}$ is 0. It remains to show **Game 2** is indistinguishable from **Game 1**. We show this by a reduction from the DLWE problem.

Reduction from DLWE. Assuming $\mathcal{A}$ has a non-negligible advantage in distinguishing Game 1 and Game 2, we use $\mathcal{A}$ to construct a DLWE algorithm $\mathcal{B}$.

- **DLWE Instance.** $\mathcal{B}$ begins by obtaining an DLWE challenge instance with two random matrices $\mathbf{A}, \mathbf{U} \in \mathbb{Z}_q^{n \times m}$ and two vector $\mathbf{c}_{in}, \mathbf{c}_{out} \in \mathbb{Z}_q^m$. Here, $\mathbf{c}_{in}$ and $\mathbf{c}_{out}$ are either random in $\mathbb{Z}_q^m$ or as distributions

$$\mathbf{c}_{in} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_0 \quad \text{and} \quad \mathbf{c}_{out} = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_1 \quad (6)$$

where $\mathbf{s} \in \mathbb{Z}_q^n$ and $\mathbf{e}_0, \mathbf{e}_1 \leftarrow \chi^m$. The algorithm $\mathcal{B}$ is intended to distinguish these two cases using $\mathcal{A}$.

- **Initial.** $\mathcal{A}$ sends a target attribute vector $\mathbf{x}^* = (x_1^*, \dots, x_l^*) \in \mathbb{Z}_q^l$ to $\mathcal{B}$.
- **Setup.** $\mathcal{B}$ generate the public parameter **pp** as in Game 1: Choose l random matrices $\mathbf{S}_1^*, \dots, \mathbf{S}_l^* \xleftarrow{\$} \{1, -1\}^{m \times m}$ and compute $\mathbf{B}_i = \mathbf{AS}_i^* - x_i^* \mathbf{G}$ for $i \in [l]$. $\mathcal{B}$ then sends $\mathcal{A}$ the public parameters, **pp**.
- **Query Phase 1.** $\mathcal{B}$ responds to $\mathcal{A}$'s queries as in Game 1.

- **Challenge.** $\mathcal{A}$ sends $\mathcal{B}$ two messages $\mu_0, \mu_1 \in \{0, 1\}^m$. Then $\mathcal{B}$ chooses a random bit $\beta \xleftarrow{\$} \{0, 1\}$, then computes and sends $\mathcal{A}$ the challenge ciphertext $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$, where $\mathbf{c}^* = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{c}_{in} \in \mathbb{Z}_q^{(l+1)m}$, and $\mathbf{c}_{out}^* = \mathbf{c}_{out} + \lfloor \frac{q}{4} \rfloor \mu_\beta \in \mathbb{Z}_q^m$. If $(\mathbf{c}_{in}, \mathbf{c}_{out})$ is hold as a random distribution, $\mathbf{c}^*$ in the challenge ciphertext is statistically close to uniform based on the standard leftover hash lemma. Since $\mathbf{c}_{out}$ is random, $\mathbf{c}_{out}^*$ is also random. Thus, the challenge ciphertext $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$ is statistically close to uniform, as in Game 2. Note that from the algorithm Enc, an encryption for $\mathbf{x}^*$ uses the matrix: $\mathbf{H}_\mathbf{x} = [\mathbf{A} | \mathbf{B}_1 + x_1^* \mathbf{G} | \dots | \mathbf{B}_l + x_l^* \mathbf{G}] = [\mathbf{A} | \mathbf{A}\mathbf{S}_1^* | \dots | \mathbf{A}\mathbf{S}_l^*]$. If $(\mathbf{c}_{in}, \mathbf{c}_{out})$ are pseudorandom distributions as in (8), we have:

$$\mathbf{c}^* = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot (\mathbf{A}^\top \mathbf{s} + \mathbf{e}_0) = \mathbf{H}_\mathbf{x}^\top \mathbf{s} + [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{e}_0$$

and $\mathbf{c}_{out}^* = \mathbf{c}_{out} + \lfloor \frac{q}{4} \rfloor \mu_\beta = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_1 + \lfloor \frac{q}{4} \rfloor \mu_\beta$. It shows that $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$ is a valid encryption of μ_β , with the same distribution as in Game 1.

- **Query Phase 2.** $\mathcal{B}$ responds to $\mathcal{A}$'s queries as in Query Phase 1.
- **Guess.** $\mathcal{A}$ guesses if it is interacting with a Game 1 or Game 2 challenger. $\mathcal{B}$ outputs its answer to the DLWE challenge based on $\mathcal{A}$'s output.

Based on the analysis above, when the LWE challenge is a random instance, $\mathcal{A}$'s view is as in Game 2. When the LWE challenge is a pseudorandom instance as in (8), $\mathcal{A}$'s view is as in Game 1. Therefore, $\mathcal{B}$'s advantage in solving the DLWE problem is at least as large as $\mathcal{A}$'s advantage in distinguishing Game 1 and Game 2, which completes the proof.

5 Asymptotic Performance

In this section, we present an asymptotic space and time complexity performance analysis for the respective key components and operations of the proposed FHABE scheme, as summarized in Table 2. As shown, the asymptotic sizes of the secret key and evaluation keys are $O(2n^2 \log^3 q)$ and $O(Nn^2 \log^3 q \log Q)$ respectively, where n and N are the dimensions of LWE and RLWE encryptions, q and Q are the modulus of LWE and RLWE encryptions. The asymptotic size of the public parameters is $O((l+3)n^2 \log^2 q)$ and the size of the fresh ciphertext is $O((l+2)n \log^2 q)$. Both of them are linear functions of the attribute vector length l . The size of evaluated ciphertexts is $O(3n \log^2 q)$.

Regarding asymptotic time performance, the homomorphic gate (i.e. NAND) computation is dominated by FHEW bootstrapping algorithm, $\text{Bootstrap}_{(\text{NAND})}$, which requires $O(n \log^2 q \log^2 Q)$ ring multiplications [21]. The decryption algorithm Dec is dominated by the key-homomorphic evaluation algorithm $\text{KeyEval}_{\text{ct}}$, which runs in $O(\kappa(d_{\mathcal{F}})ln^2 \log^2 q)$ time [6], where $\kappa(d_{\mathcal{F}})$ is a function of the number of intermediate gates, corresponding to the supported policy depth $d_{\mathcal{F}}$. The encryption algorithm Enc operates in $O(ln^2 \log q)$ time. The evaluation key generation, dominated by the bootstrapping key generation, takes $O(n \log^2 q \log Q)$

Table 2: Asymptotic space and computation performance of FHABE

Space		Time	
Component	Complexity	Type	Complexity
Public Parameters	$O((l+3)n^2 \log^2 q)$	Hom NAND	$O(n \log^2 q \log^2 Q)$
Fresh Ciphertext (Evaluated)	$O((l+2)n \log^2 q)$ $(O(3n \log^2 q))$	Encryption	$O(\ln^2 \log q)$
Secret Key	$O(2n^2 \log^3 q)$	Decryption	$O(\kappa(d_{\mathcal{F}}) \ln^2 \log^2 q)$
Evaluation Keys	$O(Nn^2 \log^3 q \log Q)$	Evaluation KeyGen	$O(n \log^2 q \log Q)$

time, including $O(2m \log q)$ RGSW encryptions, each of which involves $O(\log Q)$ ring multiplications.

6 Extension to Proxy Re-Encryption Setting

In this section, we present an unbounded multi-hop attribute-based fully homomorphic proxy re-encryption (AB-FHPRE) scheme, which serves as an extension of our unbounded FHABE into the proxy re-encryption setting.

Attribute-based PRE (AB-PRE) [34, 23, 43] is a generalization of PRE, enabling fine-grained access control on both ciphertext and delegation. Specifically, (key-policy) AB-PRE allows a proxy to transform ciphertexts under some policy f_1 , to ciphertexts under another policy f_2 , realizing many-to-many encrypted data sharing.

In our extension, AB-FHPRE incorporates FHABE and PRE, simultaneously supporting fine-grained access control, homomorphic computations of circuits with unbounded depth, and unbounded hops of re-encryptions.

Definition 5 (AB-FHPRE) *An unbounded multi-hop attribute-based fully homomorphic proxy re-encryption scheme extends the algorithms of the FHABE = (Setup, KeyGen, Enc, Dec, Eval) scheme mentioned above by incorporating two additional algorithms, ReKeyGen and ReEnc, defined as follows:*

- $\text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, f_1, f_2) \rightarrow \text{rek}_{f_1 \rightarrow f_2}$: takes a secret key sk_{f_1} and two policies, f_1, f_2 , outputs a re-encryption key $\text{rek}_{f_1 \rightarrow f_2}$ from f_1 to f_2 . This algorithm is run by the delegator, and the output will be sent to the proxy.
- $\text{ReEnc}(\text{pp}, \text{rek}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}) \rightarrow \hat{\text{ct}}_{f_2}^{(r)}$: takes a re-encryption key $\text{rek}_{f_1 \rightarrow f_2}$, a public key pk_{f_2} , and a ciphertext ct from one of the cases: (1) a fresh ciphertext $\text{ct}_{\mathbf{x}}$ such that $f_1(\mathbf{x}) = 0$; or (2) a re-encrypted or evaluated ciphertext ct_{f_1} under f_1 , outputs a re-encrypted ciphertext, $\hat{\text{ct}}_{f_2}^{(r)}$, under the policy f_2 . This algorithm is run by the proxy, and the output is sent to the delegator.

Besides the correctness requirements described in Section 2.9, an AB-FHPRE scheme additionally has the following requirement:

- For $(pk_{f_1}, sk_{f_1}) \leftarrow \text{KeyGen}(pp, msk, f_1)$, $(pk_{f_2}, sk_{f_2}) \leftarrow \text{KeyGen}(pp, f_2)$, a re-encryption key $rek_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(pp, sk_{f_1}, f_1, f_2)$, and a fresh, evaluated, or re-encrypted ciphertext ct for a message $\mu \in \{0, 1\}^*$ under f_1 , it holds:

$$\Pr[\text{Dec}(pp, sk_{f_2}, \hat{ct}_{f_2}^{(r)}) = \mu] \geq 1 - \text{negl}(\lambda)$$

where $\hat{ct}_{f_2}^{(r)} \leftarrow \text{ReEnc}(pp, pk_{f_2}, rek_{f_1 \rightarrow f_2}, ct)$ is under the policy f_2 .

Accordingly, the construction of AB-FHPRE includes the additional algorithms, ReKeyGen and ReEnc , as follows:

- $\text{ReKeyGen}(pp, sk_{f_1}, f_1, f_2) \rightarrow rek_{f_1 \rightarrow f_2}$: On input $sk_{f_1} = \mathbf{R}_{f_1} \in \mathbb{Z}_q^{2m \times m}$, and two policies f_1 and f_2 , do:
 1. Select an attribute vector $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
 2. Construct matrices $\bar{\mathbf{M}}_1^{(r)} = [\mathbf{A}|\mathbf{B}_1 + y_1\mathbf{G}|\dots|\mathbf{B}_l + y_l\mathbf{G}]^\top \mathbf{E}_1^{(r)} + \bar{\mathbf{E}}_2^{(r)}$ and $\mathbf{M}_2 = \mathbf{U}^\top \mathbf{E}_1^{(r)} + \mathbf{E}_3^{(r)} - \text{P2}(\mathbf{R}_{f_1})^\top$, where $\mathbf{E}_1^{(r)} \xleftarrow{\$} \mathbb{Z}_q^{n \times 2md}$, $\bar{\mathbf{E}}_2^{(r)} \leftarrow \chi^{(l+1)m \times 2md}$, and $\mathbf{E}_3^{(r)} \leftarrow \chi^{m \times 2md}$.
 3. Evaluate $\mathbf{D}_{f_2, \mathbf{y}} \leftarrow \text{KeyEval}_D(f_2, \mathbf{y}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$ and compute $\mathbf{M}_1^{(r)} = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f_2, \mathbf{y}} \end{bmatrix}^\top \cdot \bar{\mathbf{M}}_1^{(r)} = [\mathbf{A}|\mathbf{B}_{f_2}]^\top \mathbf{E}_1^{(r)} + \mathbf{E}_2^{(r)}$, for some error $\mathbf{E}_2^{(r)}$ satisfying $\|\mathbf{E}_2\|_\infty < \alpha_{\mathcal{F}} \cdot B_\chi$.
 4. Output the re-encryption key: $rek_{f_1 \rightarrow f_2} = \begin{bmatrix} \mathbf{M}_1^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{M}_2^{(r)} & \mathbf{I}_{m \times m} \end{bmatrix}$.
- $\text{ReEnc}(pp, rek_{f_1 \rightarrow f_2}, pk_{f_2}, ct) \rightarrow \hat{ct}_{f_2}^{(r)}$: takes a re-encryption key from f_1 to f_2 , $rek_{f_1 \rightarrow f_2}$, a ciphertext ct , and a public key pk_{f_2} for f_2 , do:
 1. If ct is a fresh ciphertext under $\mathbf{x}$, as $ct_{\mathbf{x}} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$, output $\perp$ if $f_1(\mathbf{x}) \neq 0$; Otherwise, evaluate $\mathbf{c}_{f_1} \leftarrow \text{KeyEval}_{ct}(f_1, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i=1}^l)$ and construct the functioned ciphertext $\hat{ct}_{f_1} = (\hat{\mathbf{c}}_{f_1}, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$, where $\hat{\mathbf{c}}_{f_1} = [\mathbf{c}_{in} \parallel \mathbf{c}_{f_1}]$; If ct is an evaluated or re-encrypted ciphertext for f , output $\perp$ if $f \neq f_1$; Otherwise, let ct be $\hat{ct}_{f_1} = (\hat{\mathbf{c}}_{f_1} = [\mathbf{c}_{in} \parallel \mathbf{c}_{f_1}], \mathbf{c}_{out})$.
 2. Compute the 1-hop re-encrypted ciphertext $\bar{ct}_{f_2}^{(r)} = rek_{f_1 \rightarrow f_2} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1}) \\ \mathbf{c}_{out} \end{bmatrix}$.
 3. Output the final ciphertext $\hat{ct}_{f_2}^{(r)} \leftarrow \text{Eval.AND}(pp, pk_{f_2}, \bar{ct}_{f_2}^{(r)}, \bar{ct}_{f_2}^{(r)})$.

The step 2 in ReEnc produce an 1-hop re-encryption $\bar{ct}_{f_2}^{(r)}$ for f_2 with accumulated error, such that it cannot be further computed or re-encrypted. So the last step is a homomorphic AND computation that consists of a simple addition and a bootstrapping algorithm, $\text{Bootstrap}_{\text{AND}}$, over the re-encrypted ciphertext $\bar{ct}_{f_2}^{(r)}$ and itself. Since AND operation over a binary bit and itself preserves the same message, this step is intended to refresh the re-encrypted ciphertext to a lower error state such that it is correct for the next re-encryption, achieving the goal of unbounded operations.

Correctness of AB-FHPRE. The last step of ReEnc is a homomorphic AND computation intended to reduce the error. Thus, a re-encrypted ciphertext output by ReEnc is also considered an evaluated ciphertext whose correctness has been already guaranteed by the relations (3), (4) and (5). Therefore, an AB-FHPRE scheme suffices to additionally consider the correctness of the re-encryption of any evaluated ciphertext.

- Supposed the re-encrypted ciphertext is generated from a functioned ciphertext after a computation, $\hat{\mathbf{ct}}_{f_1}^{(g)} = (\hat{\mathbf{c}}_{f_1} = [\mathbf{c}_{in} \parallel \mathbf{c}_{f_1}], \mathbf{c}_{out})$ with an error $\mathbf{e}^{(g)}$ holding $\|\mathbf{e}^{(g)}\|_\infty \leq B^{(g)} = 2m^2d \cdot B_\chi + \delta + 2m\sigma \cdot 2m^2d \cdot \alpha_{\mathcal{F}} B_\chi$ from (5). The intermediate 1-hop re-encrypted ciphertext $\bar{\mathbf{ct}}_{f_2}^{(r)}$ is generated as:

$$\bar{\mathbf{ct}}_{f_2}^{(r)} = \begin{bmatrix} [\mathbf{A} \parallel \mathbf{B}_{f_2}]^\top \mathbf{E}_1^{(r)} + \mathbf{E}_2^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{U}^\top \mathbf{E}_1^{(r)} + \mathbf{E}_3^{(r)} - \mathbf{P}2(\bar{\mathbf{R}}_{f_1})^\top & \mathbf{I}_{m \times m} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1}) \\ \mathbf{c}_{out} \end{bmatrix}$$

Let $\tilde{\mathbf{s}}^{(r)} = \mathbf{E}_1^{(r)} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1})$, $\tilde{\mathbf{e}}^{(r)} = \mathbf{E}_2^{(r)} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1})$, and $\tilde{\mathbf{e}}_{out}^{(r)} = \mathbf{E}_3^{(r)} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1}) + \mathbf{e}^{(g)}$. We have $\bar{\mathbf{ct}}_{f_2}^{(r)} = ([\mathbf{A} \parallel \mathbf{B}_{f_2}]^\top \tilde{\mathbf{s}}^{(r)} + \tilde{\mathbf{e}}^{(r)}, \mathbf{U}^\top \tilde{\mathbf{s}}^{(r)} + \tilde{\mathbf{e}}_{out}^{(r)} + \lfloor q/4 \rfloor \cdot \boldsymbol{\mu})$, where the decryption error is $\tilde{\mathbf{e}}_{out}^{(r)} - \mathbf{R}_{f_2}^\top \tilde{\mathbf{e}}^{(r)}$. Thus, to ensure $\bar{\mathbf{ct}}_{f_2}^{(r)}$ is correct as an input ciphertext to Eval.AND, the parameters must be chosen satisfying:

$$\begin{aligned} \|\tilde{\mathbf{e}}_{out}^{(r)} - \mathbf{R}_{f_2}^\top \tilde{\mathbf{e}}^{(r)}\| &\leq \|\mathbf{E}_3^{(r)} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1})\| + \|\mathbf{e}^{(g)}\| + \|\mathbf{R}_{f_2}^\top \mathbf{E}_2^{(r)} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1})\| \\ &\leq 2mdB_\chi + B^{(g)} + 2m\sigma \cdot 2md\alpha_{\mathcal{F}} B_\chi < \frac{q}{16} \end{aligned} \quad (7)$$

In summary, we say an AB-FHPRE scheme is correct if the chosen parameters satisfy the conditions (3), (4), (5), (7).

Security. We claim that the AB-FHPRE scheme given above is selective secure under the honest re-encryption attack (HRA). The security model and security proof follow the framework of [43]. The main difference is the challenger will answer the public key $\mathbf{pk}_f$ to queries of f satisfying $f(\mathbf{x}^*) = 0$ as in Section 4. For conciseness, we defer the formal security proof to our full version.

7 Conclusions and Future Works

To address the limitation of the existing HABE schemes, which only allow for the homomorphic evaluation of circuits with a prior bounded depth, we introduce a novel bootstrappable FHABE scheme. This scheme supports computation of circuits with unbounded depth, making it suitable for a broader range of real-world applications. Additionally, in contrast to the prior HABE schemes, our scheme provides a more lightweight ciphertext, and is proven to be secure under the standard model. Extending this scheme to a PRE setting, we also propose the first lattice-based multi-hop attribute-based fully homomorphic proxy re-encryption scheme, which allows for an unbounded number of re-encryptions.

Several interesting directions for future research arise from our work. Although our scheme supports homomorphic evaluation of unbounded circuits, the supported policy functions are still limited to a bounded depth. It would be interesting to explore a FHABE scheme with unbounded policy functions. Another intriguing question is whether we can develop an unbounded ciphertext-policy FHABE scheme.

Acknowledgements. This work is supported by the Research Grant Council of Hong Kong (Project No. C1029-22G), The Hong Kong Polytechnic University (Project No. P0046340), the Ministry of Education in Singapore under Grants RG102/24, Singapore Ministry of Education Academic Research Fund Tier 2 Grant MOE-T2EP20223-0016, Singapore Ministry of Education Academic Research Fund Tier 2 Grant MOE-T2EP20223-0016, the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority. Jian Weng was supported in part by the National Natural Science Foundation of China (Nos. 62332007, U22B2028), the Science and Technology Major Project of Tibetan Autonomous Region of China (No. XZ202201ZD0006G), Open Research Fund of Machine Learning and Cyber Security Interdiscipline Research Engineering Center of Jiangsu Province (No. SDGC2131), National Joint Engineering Research Center of Network Security Detection and Protection Technology, Guangdong Key Laboratory of Data Security and Privacy Preserving, Guangdong Hong Kong Joint Laboratory for Data Security and Privacy Protection, and Engineering Research Center of Trustworthy AI, Ministry of Education.

References

1. Agrawal, S., Boneh, D., Boyen, X.: Efficient lattice (h) ibe in the standard model. In: *Advances in Cryptology—EUROCRYPT 2010*. pp. 553–572. Springer (2010)
2. Alwen, J., Peikert, C.: Generating shorter bases for hard random lattices. *Theory of Computing Systems* **48**, 535–553 (2011)
3. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with CKKS. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024*. LNCS, vol. 15484, pp. 330–360. Springer (2024). https://doi.org/10.1007/978-981-96-0875-1_11
4. Blum, A., Furst, M., Kearns, M., Lipton, R.J.: Cryptographic primitives based on hard learning problems. In: *Annual International Cryptology Conference*. pp. 278–291. Springer (1993)
5. Boneh, D., Gennaro, R., Goldfeder, S., Jain, A., Kim, S., Rasmussen, P.M.R., Sahai, A.: Threshold cryptosystems from threshold fully homomorphic encryption. In: *Advances in Cryptology - CRYPTO 2018*. LNCS, vol. 10991, pp. 565–596. Springer (2018). https://doi.org/10.1007/978-3-319-96884-1_19

6. Boneh, D., Gentry, C., Gorbunov, S., Halevi, S., Nikolaenko, V., Segev, G., Vaikuntanathan, V., Vinayagamurthy, D.: Fully key-homomorphic encryption, arithmetic circuit abe and compact garbled circuits. In: *Advances in Cryptology–EUROCRYPT 2014*. pp. 533–556. Springer (2014)
7. Bossuat, J., Mouchet, C., Troncoso-Pastoriza, J.R., Hubaux, J.: Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In: *Advances in Cryptology - EUROCRYPT 2021*. LNCS, vol. 12696, pp. 587–617. Springer (2021). https://doi.org/10.1007/978-3-030-77870-5_21
8. Boudgoust, K., Scholl, P.: Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In: *Advances in Cryptology - ASIACRYPT 2023*. LNCS, vol. 14438, pp. 371–404. Springer (2023). https://doi.org/10.1007/978-981-99-8721-4_12
9. Boyen, X.: Attribute-based functional encryption on lattices. In: *Theory of cryptography conference*. pp. 122–142. Springer (2013)
10. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical gapsvp. In: *Annual cryptology conference*. pp. 868–886. Springer (2012)
11. Brakerski, Z., Cash, D., Tsabary, R., Wee, H.: Targeted homomorphic attribute-based encryption. In: *Theory of Cryptography: TCC 2016-B, Part II*. pp. 330–360. Springer (2016)
12. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory* **6**(3), 13:1–13:36 (2014)
13. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) LWE. *Electron. Colloquium Comput. Complex.* **TR11-109** (2011)
14. Brakerski, Z., Vaikuntanathan, V.: Circuit-abe from lwe: unbounded attributes and semi-adaptive security. In: *Annual International Cryptology Conference*. pp. 363–384. Springer (2016)
15. Cash, D., Hofheinz, D., Kiltz, E., Peikert, C.: Bonsai trees, or how to delegate a lattice basis. In: Gilbert, H. (ed.) *Advances in Cryptology - EUROCRYPT 2010*. LNCS, vol. 6110, pp. 523–552. Springer (2010). https://doi.org/10.1007/978-3-642-13190-5_27
16. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: A full RNS variant of approximate homomorphic encryption. In: Cid, C., Jr., M.J.J. (eds.) *Selected Areas in Cryptography - SAC 2018*. LNCS, vol. 11349, pp. 347–368. Springer (2018). https://doi.org/10.1007/978-3-030-10970-7_16
17. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: *Advances in Cryptology - ASIACRYPT 2017*. vol. 10624, pp. 409–437 (2017)
18. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: *Advances in Cryptology - ASIACRYPT 2016*. vol. 10031, pp. 3–33 (2016)
19. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.* **33**(1), 34–91 (2020). <https://doi.org/10.1007/S00145-019-09319-X>
20. Clear, M., Goldrick, C.M.: Attribute-based fully homomorphic encryption with a bounded number of inputs. *International Journal of Applied Cryptography* **3**(4), 363–376 (2017)
21. Ducas, L., Micciancio, D.: FHEW: bootstrapping homomorphic encryption in less than a second. In: *Advances in Cryptology - EUROCRYPT 2015*. vol. 9056, pp. 617–640 (2015)

22. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.* p. 144 (2012)
23. Ge, C., Susilo, W., Wang, J., Huang, Z., Fang, L., Ren, Y.: A key-policy attribute-based proxy re-encryption without random oracles. *Comput. J.* **59**(7), 970–982 (2016). <https://doi.org/10.1093/comjnl/bxv100>
24. Geelen, R., Vercauteren, F.: Bootstrapping for BGV and BFV revisited. *J. Cryptol.* **36**(2), 12 (2023). <https://doi.org/10.1007/S00145-023-09454-6>
25. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009*. pp. 169–178 (2009)
26. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: Dwork, C. (ed.) *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*. pp. 197–206. ACM (2008). <https://doi.org/10.1145/1374376.1374407>
27. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: *Advances in Cryptology—CRYPTO 2013*. pp. 75–92 (2013)
28. Gorbunov, S., Vaikuntanathan, V., Wee, H.: Attribute-based encryption for circuits. In: *Proceedings of the forty-fifth annual ACM symposium on Theory of Computing*. pp. 545–554 (2013)
29. Halevi, S., Polyakov, Y., Shoup, V.: An improved RNS variant of the BFV homomorphic encryption scheme. In: Matsui, M. (ed.) *Topics in Cryptology - CT-RSA 2019*. LNCS, vol. 11405, pp. 83–105. Springer (2019). https://doi.org/10.1007/978-3-030-12612-4_5
30. Hiromasa, R., Kawai, Y.: Dynamic multi target homomorphic attribute-based encryption. In: *Cryptography and Coding: 16th IMA International Conference, IMACC 2017*. pp. 25–43. Springer (2017)
31. Kwak, H., Min, S., Song, Y.: Towards practical multi-key TFHE: parallelizable, key-compatible, quasi-linear complexity. In: *Public-Key Cryptography - PKC 2024*. LNCS, vol. 14604, pp. 354–385. Springer (2024). https://doi.org/10.1007/978-3-031-57728-4_12
32. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: *Advances in Cryptology - EUROCRYPT 2023*. LNCS, vol. 14006, pp. 227–256 (2023). https://doi.org/10.1007/978-3-031-30620-4_8
33. Li, H., Lin, H., Luo, J.: ABE for circuits with constant-size secret keys and adaptive security. In: Kiltz, E., Vaikuntanathan, V. (eds.) *Theory of Cryptography - TCC 2022*. LNCS, vol. 13747, pp. 680–710. Springer (2022). https://doi.org/10.1007/978-3-031-22318-1_24
34. Liang, X., Cao, Z., Lin, H., Shao, J.: Attribute based proxy re-encryption with delegating capabilities. In: *Proceedings of the 4th international symposium on information, computer, and communications security*. pp. 276–286 (2009)
35. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: *EUROCRYPT 2010*. pp. 1–23. Springer (2010)
36. Micciancio, D., Peikert, C.: Trapdoors for lattices: Simpler, tighter, faster, smaller. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 700–718. Springer (2012)
37. Micciancio, D., Polyakov, Y.: Bootstrapping in fhew-like cryptosystems. In: *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. pp. 17–28 (2021)

38. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key fhe. In: Advances in Cryptology–EUROCRYPT 2016. pp. 735–763. Springer (2016)
39. Okada, H., Player, R., Pohmann, S.: Homomorphic polynomial evaluation using galois structure and applications to BFV bootstrapping. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology - ASIACRYPT 2023. LNCS, vol. 14443, pp. 69–100. Springer (2023). https://doi.org/10.1007/978-981-99-8736-8_3
40. Peikert, C., Shiehian, S.: Multi-key fhe from lwe, revisited. In: Theory of cryptography conference. pp. 217–238. Springer (2016)
41. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Proceedings of the 37th Annual ACM Symposium on Theory of Computing. pp. 84–93 (2005)
42. Sahai, A., Waters, B.: Fuzzy identity-based encryption. In: Advances in Cryptology–EUROCRYPT 2005. pp. 457–473. Springer (2005)
43. Susilo, W., Dutta, P., Duong, D.H., Roy, P.S.: Lattice-based hra-secure attribute-based proxy re-encryption in standard model. In: Computer Security–ESORICS 2021. pp. 169–191. Springer (2021)
44. Van Dijk, M., Gentry, C., Halevi, S., Vaikuntanathan, V.: Fully homomorphic encryption over the integers. In: Advances in Cryptology–EUROCRYPT 2010. pp. 24–43. Springer (2010)
45. Xiang, B., Zhang, J., Deng, Y., Dai, Y., Feng, D.: Fast blind rotation for bootstrapping fhcs. In: Advances in Cryptology - CRYPTO 2023. LNCS, vol. 14084, pp. 3–36

A Multiplication Operations between RLWE and RGSW

For the RLWE' and RGSW encryptions as:

- $\text{RLWE}'_z(m) = (\text{RLWE}_z(m), \text{RLWE}_z(B_g m), \dots, \text{RLWE}_z(B_g^{d_g-1} m)) \in \mathcal{R}_Q^{2d_g}$
- $\text{RGSW}_z(m) = (\text{RLWE}'_z(-z \cdot m), \text{RLWE}'_z(m)) \in \mathcal{R}_Q^{2d_g \times 2}$

The operations between them are defined as follows: The scalar multiplication between an element in $\mathcal{R}_Q$ and a RLWE' ciphertext is defined as:

$$\odot : \mathcal{R}_Q \times \text{RLWE}' \rightarrow \text{RLWE}$$

$$\begin{aligned} r \odot \text{RLWE}'_z(m) &= \langle (r_0, \dots, r_{d_g-1}), (\text{RLWE}_z(m), \dots, \text{RLWE}_z(B_g^{d_g-1} m)) \rangle \\ &= \sum_{i=0}^{d_g-1} r_i \cdot \text{RLWE}_z(B_g^i \cdot m) = \text{RLWE}_z(r \cdot m) \in \mathcal{R}_Q^2 \end{aligned}$$

The error term of output RLWE encryption is $\sum_{i=0}^{d_g-1} r_i \cdot e_i$ where e_i is the error of $\text{RLWE}_z(B_g^i \cdot m)$. The multiplication between a RLWE ciphertext and a RGSW ciphertext is defined as:

$$\diamond : \text{RLWE} \times \text{RGSW} \rightarrow \text{RLWE}$$

$$\begin{aligned}
\text{RLWE}_z(m_1) \diamond \text{RGSW}_z(m_2) &= (a, b) \diamond (\text{RLWE}'_z(-z \cdot m_2), \text{RLWE}'_z(m_2)) \\
&= a \odot \text{RLWE}'_z(-z \cdot m_2) + b \odot \text{RLWE}'_z(m_2) \\
&= \text{RLWE}_z(-z \cdot a \cdot m_2) + \text{RLWE}_z(b \cdot m_2) \\
&= \text{RLWE}_z((b - z \cdot a) \cdot m_2) \\
&= \text{RLWE}_z(m_1 m_2 + e_1 m_2) \in \mathcal{R}_Q^2
\end{aligned}$$

The error term is related to the size of m_2 , so to make this term small for correctness of bootstrapping, typically we set $m_2 = \pm X^v$ for some v so that $\|e_1 m_2\| = \|e_1\|$ and $\text{RLWE}_z(m_1) \diamond \text{RGSW}_z(m_2) \approx \text{RLWE}_z(m_1 m_2)$.

B More details of FHEW Bootstrapping

In this section we introduce more details about the FHEW bootstrapping. Note that the decryption of a LWE ciphertext $(\mathbf{a}, b)$ is processed by first computing $m' = b - \langle \mathbf{a}, \mathbf{s} \rangle$ followed by a rounding $m = \lfloor (t/q)m' \rfloor$. The goal of the bootstrapping is to homomorphically compute this decryption given an encryption of the secret key $\mathbf{s}$, and output a new LWE encryption of m with a small noise.

Key Switching. Key switching converts a LWE encryption under a key $\mathbf{z} \in \mathbb{Z}_q^N$ to a LWE encryption under a different key $\mathbf{s} \in \mathbb{Z}_q^n$, maintaining the same underlying message. The key switching includes two algorithms:

- $\text{KskGen}(\mathbf{z}, \mathbf{s}, B_{\text{ks}})$: takes a key switch base B_{ks} , outputs $\text{ksk} = \{\mathbf{k}_{h,i,j} = \text{LWE}_{\mathbf{s}}^{q/q}(hz_i B_{\text{ks}}^j)\}$ for $h = 0, \dots, B_{\text{ks}} - 1, i = 1, \dots, N, j = 0, \dots, d_{\text{ks}} - 1$ where $d_{\text{ks}} = \lceil \log_{B_{\text{ks}}} q \rceil$.
- $\text{KeySwitch}((\mathbf{a}, b), \text{ksk})$: takes a LWE encryption $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{z}}^{t/q}(m)$ for some m , for each i , compute $a_{i,j}$ such that $-a_i = \sum_{j=0}^{d_{\text{ks}}-1} a_{i,j} B_{\text{ks}}^j$, then compute $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s}}^{t/q}(m) = (\mathbf{0}, b) + \sum_{i,j} \mathbf{k}_{a_{i,j}, i, j}$.

Fact 2 (From Lemma 6 in [21]) *Given a ciphertext $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{z}}^{t/q}(m)$ with sub-Gaussian error of parameter α and a key switch key $\text{ksk} = \{\text{LWE}_{\mathbf{s}}^{q/q}(hz_i B_{\text{ks}}^j)\}$ with sub-Gaussian error of parameter σ , the key switching outputs an encryption $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s}}^{t/q}(m)$ with sub-Gaussian error of parameter $\sqrt{\alpha^2 + Nd_{\text{ks}}\sigma^2}$.*

Modulus Switching. Modulus switching converts a LWE encryption under a modulus Q , to a LWE encryption under another modulus q , maintaining the same underlying message. This operation uses a scaled rounding function $[x]_{Q/q} = \lfloor (xq/Q) \rfloor$ for some x . For $(\mathbf{a} = (a_1, \dots, a_n), b) \in \text{LWE}_{\mathbf{s}}^{t/Q}(m)$, modulus switching proceeds with the following algorithm:

$$\text{ModSwitch}(\mathbf{a}, b) = ([a_1]_{Q/q}, \dots, [a_n]_{Q/q}, [b]_{Q/q})$$

Fact 3 (From Lemma 5 in [21]) *According to the central limit heuristic, for $\mathbf{s} \in \mathbb{Z}_q, m \in \mathbb{Z}_t$, and a ciphertext $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}}^{t/Q}(m)$ with sub-Gaussian error of parameter σ , the output, $\text{ModSwitch}((\mathbf{a}, b))$, is in $\text{LWE}_{\mathbf{s}}^{t/q}(m)$ with sub-Gaussian error of parameter $\sqrt{(q\sigma/Q)^2 + (\|\mathbf{s}\|^2 + 1)/12}$.*

Homomorphic Accumulator. The FHEW bootstrapping will use a Homomorphic Accumulator scheme consists of the following operations:

- $\text{Init}(\text{ACC} \stackrel{f}{\leftarrow} v)$: On input $v \in \mathbb{Z}_q$, set $\text{ACC} = (0, v_f) \in \text{RLWE}_z(v_f)$, where $v_f = \sum_{i=0}^{N-1} f(v-i) \cdot X^{i \cdot 2N/q}$ and $f(\cdot)$ is a mapping based on computations.
- $\text{Update}(\text{ACC} \stackrel{+}{\leftarrow} \mathbf{C})$: On input the current ACC and a GSW ciphertext $\mathbf{C} \in \mathcal{R}_Q^{2d_g \times 2}$, update: $\text{ACC} = \text{ACC} \diamond \mathbf{C}$.
- $\text{ExtractLWE}(\text{ACC})$: On input a RLWE ciphertext $\text{ACC} = (a, b) \in \text{RLWE}_z(m)$, output: $(\mathbf{a}, b_0) \in \text{LWE}_{\mathbf{z}}^{t/Q}(m)$ where $\mathbf{a} = (a_0, \dots, a_{N-1})$ is the coefficients vector of $a \in \mathcal{R}_Q$, b_0 is the constant term of b , and $\mathbf{z} = (z_0, -z_{q/2-1}, \dots, -z_1)$.

Here, we adapt the definition of mapping function f as in [37]. The target of the mapping f is to ensure the correctness of the extraction algorithm ExtractLWE . Consider the NAND computation of $c_1 = (\mathbf{a}_1, b_1) \in \text{LWE}_{\mathbf{s}}^{4/q}(m_1, q/16)$, $c_2 = (\mathbf{a}_2, b_2) \in \text{LWE}_{\mathbf{s}}^{4/q}(m_2, q/16)$. By computing $(c_1 + c_2)$, we obtain the ciphertext $c = (\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}}^{4/q} \in (m_1 + m_2, q/8)$. The decryption computes $b - \langle \mathbf{a}, \mathbf{s} \rangle = (q/4)(m_1 + m_2) + (e_1 + e_2)$ where $|e_1 + e_2| < q/8$ ($|e_i| < q/16$). For the correctness of the final results, the function f is intended to map $0 \rightarrow 1, 1 \rightarrow 1, 2 \rightarrow 0, 3 \rightarrow 0$ with the scalar $q/4$. Thus, f maps $x \in (0, q/4) \pm q/8$ to $q/4$ and maps $x \in (q/2, 3q/4) \pm q/8$ to 0. To satisfy the structure requirement $f(x + q/2) = -f(x)$, the function f is shifted to map $x \in [-q/8, 3q/8) \rightarrow q/8$ and $x \in [3q/8, 7q/8) \rightarrow -q/8$. The $q/8$ will be added after extraction for correct mapping. Table 3 shows the computations and mappings for various logic gate computations. The bootstrapping procedure for a LWE encryption requires a

Table 3: Computations and mappings for logic gates.

Gate	Computation	maps to $q/8$	maps to $-q/8$
NAND	$c_1 + c_2$	$[-q/8, 3q/8)$	$[3q/8, 7q/8)$
AND	$c_1 + c_2$	$[3q/8, 7q/8)$	$[-q/8, 3q/8)$
OR	$c_1 + c_2$	$[q/8, 5q/8)$	$[-3q/8, q/8)$
XOR	$2(c_1 - c_2)$	$[q/8, 5q/8)$	$[-3q/8, q/8)$

bootstrapping key btk which is a GSW-type encryption of the secret key $\mathbf{s}$ as:

$$\text{btk} = \{\text{RGSW}_z(X^{v \cdot s_i \cdot B_r^j})\}_{i,j,v} \text{ where } i \in [n], j \in [0, \dots, d_r - 1], v \in [0, B_r - 1]$$

For LWE parameters n and q , a bootstrapping base B_r , a key switching base B_{ks} , a Gadget base B_g , a RLWE/RGSW dimension $N = q/2$, and a RLWE/RGSW ciphertext modulus Q , the bootstrapping algorithm for NAND gate evaluation is presented as Algorithm 1.

Theorem 2 (From [21]). *Assuming the hardness of $\text{RLWE}_{\mathcal{R}, Q, \bar{\chi}}$ and based on the central limit heuristic, the error of the output from the algorithm Bootstrap*

Algorithm 1 Bootstrap($\text{btk}_s, \text{ksk}_s, \mathbf{c} = (\mathbf{a}, b)$)(NAND)

Input: $\text{btk}_s = \{\mathbf{Z}_{h,i,j}\}$, $\text{ksk}_s = \{\mathbf{k}_{u,v,w}\}$, and $\mathbf{c} = (\mathbf{a}, b) = (\mathbf{a}_0 + b_0) + (\mathbf{a}_1, b_1)$ $\triangleright$
 $\mathbf{Z}_{h,i,j} = \text{RGSW}_z(X^{h \cdot s_i \cdot B_r^j})$, $\mathbf{k}_{u,v,w} = \text{LWE}_s^{q/q}(uz'_v B_{ks}^w)$, $\mathbf{z}' = (z_0, -z_{q/2-1}, \dots, -z_1)$,
and $(\mathbf{a}_i, b_i) \in \text{LWE}_s^{4/q}(m_i, q/16)$
Output: $\tilde{\mathbf{c}} \in \text{LWE}_s^{4/q}(m, \delta)$ $\triangleright m = m_0 \bar{\wedge} m_1$

- 1: Let $p = 2N/q$
- 2: Init($\text{ACC} \xleftarrow{f} b$): $\text{ACC} = (0, b') : b' = \sum_{j=0}^{N-1} b'_j \cdot X^{j \cdot p \pmod{q/2}}$ $\triangleright b'_j = -Q/8$ if $(b-j) \bmod q \in [3q/8, 7q/8]$; Otherwise $b'_j = Q/8$.
- 3: **for** $i = 0, \dots, n$
- 4: Compute the B_r -base decomposition of $-a_i = \sum_j B_r^j \cdot a_{i,j}$
- 5: **for** $j = 0, \dots, d_r - 1$ $\triangleright d_r = \lceil \log_{B_r} q \rceil$
- 6: **for** $h = 0, \dots, B_r - 1$
- 7: $\text{ACC} \xleftarrow{+} \mathbf{Z}_{h,i,j}$
- 8: Let $\mathbf{c}' = (a', b') = \text{ACC} \in \mathcal{R}_q^2$
- 9: $\mathbf{c}'' = (\mathbf{a}'', b'') = (\mathbf{a}', b'_0 + Q/8) \in \text{LWE}_z^{4/Q}(m) \leftarrow \text{ExtractLWE}(\text{ACC})$ $\triangleright$
 $\mathbf{a}' = (a'_0, \dots, a'_{q/2-1})$
- 10: $\tilde{\mathbf{c}} = (\tilde{\mathbf{a}}, \tilde{b}) \in \text{LWE}_s^{4/Q}(m) \leftarrow \text{KeySwitch}((\mathbf{a}'', b''), \text{ksk}_s)$
- 11: $\tilde{\mathbf{c}} = (\tilde{\mathbf{a}}, \tilde{b}) \in \text{LWE}_s^{4/q}(m) \leftarrow \text{ModSwitch}_{Q/q}((\tilde{\mathbf{a}}, \tilde{b}))$
- 12: **return** $\tilde{\mathbf{c}} \in \text{LWE}_s^{4/q}(m, \delta)$

has a Gaussian distribution with parameter δ' and bound $\delta = \delta'/\sqrt{2\pi}$ for :

$$\delta' = \sqrt{\frac{q^2}{Q^2}(\bar{\sigma}^2 \frac{B_r^2}{12} d_r q n N(B_g + \frac{Q}{B_g^{d_g}} - 1) + \sigma^2 N d_{ks}) + \frac{\|\mathbf{s}\|^2 + 1}{12}}$$

C Security for AB-FHPRE

In this section, we define honest re-encryption attack (HRA) security and present a formal proof for our extended AB-FHPRE scheme. HRA strengthens the chosen plaintext attack (CPA) model by additionally allowing the adversary to query the re-encryption oracle on honestly generated ciphertexts. Our security proof is adapted from the HRA-secure KP-ABE framework in [43] and incorporates the techniques for answering auxiliary recovery keys during the public-key queries in the proof of our proposed FHABE.

HRA Security for AB-FHPRE. We define the security of selective secure AB-FHPRE against honest re-encryption attacks: Let $\mathcal{A}$ be an PPT adversary, $\Pi = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval}, \text{ReKeyGen}, \text{ReEnc})$ be an AB-FHPRE scheme as described above. We consider the following game, $\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-HRA}}(\lambda)$, between an adversary $\mathcal{A}$ and the challenger:

1. **Initial.** $\mathcal{A}$ outputs a target attribute vector $\mathbf{x}^* \in \mathcal{X}^l$ of its own choice.

2. **Setup.** The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^l)$ and sends pp to $\mathcal{A}$. Then the challenger initials a counter $N_{ct} = 0$, an empty key-value store $\mathcal{CT}$, and an empty set $\mathcal{TG}$.
3. **Query Phase 1.** $\mathcal{A}$ can issue polynomial times of queries to the oracles:
 - $\mathcal{O}^{\text{KeyGen}}$: On input a policy f , run $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, if $f(\mathbf{x}^*) = 0$, output pk_f ; Otherwise, output $(\text{pk}_f, \text{sk}_f)$.
 - $\mathcal{O}^{\text{ReKeyGen}}$: On input two policies f_1 and f_2 , if $f_1(\mathbf{x}^*) = 0 \wedge f_2(\mathbf{x}^*) \neq 0$, output $\perp$; Otherwise, output $\text{rek}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, f_1, f_2)$ as the re-encryption key.
 - $\mathcal{O}^{\text{Enc}}$: On input an attribute set $\mathbf{x}$ and a message μ , output ciphertext $\text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}, \mu)$, increment N_{ct} , store value $\text{ct}_{\mathbf{x}}$ to $\mathcal{CT}$ with key $(\mathbf{x}, N_{ct})$.
 - $\mathcal{O}^{\text{ReEnc}}$: On input two policies f_1 and f_2 and a key $(\mathbf{x}, j \leq N_{ct})$, if there is no value in $\mathcal{CT}$ with key $(\mathbf{x}, j)$, output $\perp$; Otherwise, let $\text{ct}_{\mathbf{x}}$ be the value, then if $f_1(\mathbf{x}) \neq 0$, output $\perp$, indicating this query is invalid; otherwise, output re-encrypted ciphertext $\text{ct}_{f_2}^{(r)} \leftarrow \text{ReEnc}(\text{pp}, \text{rek}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}_{\mathbf{x}})$.
4. **Challenge.** $\mathcal{A}$ chooses a pair of messages, μ_0, μ_1 and sends it to the challenger. The challenger randomly chooses a bit $\beta \in \{0, 1\}$, computes $\text{ct}_{\beta}^* \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \mu_{\beta})$, sends ct_{β}^* to $\mathcal{A}$ as the challenge ciphertext. Increment N_{ct} , add number N_{ct} to set $\mathcal{TG}$ and store value ct_{β}^* to $\mathcal{CT}$ with key $(\mathbf{x}^*, N_{ct})$.
5. **Query Phase 2.** $\mathcal{A}$ can continuously issue queries as in Query Phase 1 with the following exception for $\mathcal{O}^{\text{ReEnc}}$ oracle: if $f_1(\mathbf{x}^*) = 0 \wedge f_2(\mathbf{x}^*) \neq 0 \wedge j \in \mathcal{TG}$, output $\perp$.
6. **Guess.** $\mathcal{A}$ outputs a bit β' . The game outputs 1 if $\beta' = \beta$; Otherwise, the game outputs 0.

The advantage of the HRA adversary in above game is defined as:

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{IND-HRA}}(\lambda) := |\Pr[\beta' = \beta] - 1/2|$$

Definition 6 An AB-FHPRE scheme is selective IND-HRA secure if for any PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\mathcal{A}, \Pi}^{\text{IND-HRA}}(\lambda) \leq \text{negl}(\lambda)$.

We note that the key difference between the HRA and CPA security models lies in the adversary's additional access to a re-encryption oracle, which responds to queries involving honestly generated ciphertexts. In particular, with only access to the oracles $\mathcal{O}^{\text{KeyGen}}$ and $\mathcal{O}^{\text{ReKeyGen}}$, the adversary cannot obtain a re-encryption key $\text{rek}_{f_1 \rightarrow f_2}$ without violating security. This enhanced property in the HRA model is achieved by the following simulation algorithms [43]:

- **Setup_{sim}**($1^\lambda, 1^l$): Choose the parameters as in Setup, do:
 1. Choose a random matrix $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$.
 2. Choose l random matrices as $\mathbf{S}_i^* \xleftarrow{\$} \{1, -1\}^{m \times m}$ for $i \in [l]$.
 3. Compute $\mathbf{B}_i = \mathbf{A}\mathbf{S}_i^* - x_i^*\mathbf{G}$ for $i \in [l]$.
 4. Choose a random matrix $\mathbf{R}_{\mathbf{x}^*} \xleftarrow{\$} \mathbb{Z}_q^{(l+1)m \times m}$, construct $\mathbf{U} = [\mathbf{A} | \mathbf{B}_i + x_i^*\mathbf{G} | \dots | \mathbf{B}_l + x_l^*\mathbf{G}] \cdot \mathbf{R}_{\mathbf{x}^*} = [\mathbf{A} | \mathbf{A}\mathbf{S}_1^* | \dots | \mathbf{A}\mathbf{S}_l^*] \cdot \mathbf{R}_{\mathbf{x}^*}$.

5. Output the public parameter $\text{pp} = (\text{pp}^{\text{LWE}}, \text{pp}^{\text{RLWE}}, \mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U}, \mathbf{G})$.
- $\text{ReKeyGen}_{\text{sim}}(\text{pp}, f_1, f_2)$ (where $f_1(\mathbf{x}^*) = 0$): run as follows:
 1. Select an attribute vector $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
 2. If $f_2(\mathbf{x}^*) \neq 0$, output $\perp$.
 3. If $f_2(\mathbf{x}^*) = 0$, choose random matrices $\bar{\mathbf{X}}_1^{(r)} \xleftarrow{\$} \mathbb{Z}_q^{(l+1)m \times 2md}$ and $\mathbf{X}_2^{(r)} \xleftarrow{\$} \mathbb{Z}_q^{m \times 2md}$.
 4. Evaluate $\mathbf{D}_{f_2, \mathbf{y}} \leftarrow \text{KeyEval}_D(f_2, \mathbf{y}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$ and compute $\mathbf{X}_1^{(r)} = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f_2, \mathbf{y}} \end{bmatrix}^\top \cdot \bar{\mathbf{X}}_1^{(r)}$.
 5. Output the simulated re-encryption key: $\text{rek}_{f_1 \rightarrow f_2}^{(\text{sim})} = \begin{bmatrix} \mathbf{X}_1^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{X}_2^{(r)} & \mathbf{I}_{m \times m} \end{bmatrix}$.
- $\text{ReEnc}_{\text{sim-1}}(\text{pp}, f_1, f_2, (\mathbf{x}^*, j))$: Let $\text{ct}_{\mathbf{x}^*} = (\mathbf{c} = [\mathbf{c}_{in} | \mathbf{c}_1 | \dots | \mathbf{c}_l], \mathbf{c}_{out})$ be the value of key $(\mathbf{x}^*, j)$, do as follows:
 1. If $f_1(\mathbf{x}^*) \neq 0$, output $\perp$.
 2. If $f_1(\mathbf{x}^*) = 0$, do:
 - Select an attribute vector $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
 - Choose random matrix $\mathbf{R}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times (l+1)md}$, sample error matrices $\mathbf{R}_2 \leftarrow \chi^{(l+1)m \times (l+1)md}$ and $\mathbf{R}_3 \leftarrow \chi^{m \times (l+1)md}$.
 - Construct a simulated key from $\mathbf{x}^*$ to $\mathbf{y}$ as:

$$\text{rek}_{\mathbf{x}^* \rightarrow \mathbf{y}}^{(\text{sim})} = \begin{bmatrix} [\mathbf{A} | \mathbf{B}_1 + y_1 \mathbf{G} | \dots | \mathbf{B}_l + y_l \mathbf{G}]^\top \mathbf{R}_1 & \mathbf{0}_{(l+1)m \times m} \\ \mathbf{U}^\top \mathbf{R}_1 + \mathbf{R}_3 - \text{P2}(\mathbf{R}_{\mathbf{x}^*})^\top & \mathbf{I}_{m \times m} \end{bmatrix}$$
 - Compute the 1-hop re-encrypted ciphertext as:

$$\bar{\text{ct}}_{\mathbf{y}}^{(r)} = \text{rek}_{\mathbf{x}^* \rightarrow \mathbf{y}}^{(\text{sim})} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{out} \end{bmatrix} \in \mathbb{Z}_q^{(l+2)m}$$
 - Output the final ciphertext $\hat{\text{ct}}_{f_2}^{(r)} \leftarrow \text{Eval.AND}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_{\mathbf{y}}^{(r)}, \bar{\text{ct}}_{\mathbf{y}}^{(r)})$.
- $\text{ReEnc}_{\text{sim-2}}(\text{pp}, f_1, f_2, (\mathbf{x}, j))$ ($\mathbf{x} \neq \mathbf{x}^* \wedge f_1(\mathbf{x}^*) = 0$): Let $\text{ct}_{\mathbf{x}^*}$ be the value of key $(\mathbf{x}^*, j)$ as $(\mathbf{c} = [\mathbf{c}_{in} | \mathbf{c}_1 | \dots | \mathbf{c}_l], \mathbf{c}_{out})$, do as follows:
 1. If $f_1(\mathbf{x}) \neq 0$, output $\perp$.
 2. If $f_1(\mathbf{x}) = 0$, since $\mathbf{x} \neq \mathbf{x}^*$, there must be some k such that $x_k \neq x_k^*$. Assume $x_l \neq x_l^*$, do:
 - Construct $\mathbf{H}_{\mathbf{x}} = [\mathbf{A} | \mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_l + x_l \mathbf{G}] = [\mathbf{A} | \mathbf{A}\mathbf{S}_1^* + (x_1 - x_1^*)\mathbf{G} | \dots | \mathbf{A}\mathbf{S}_l^* + (x_l - x_l^*)\mathbf{G}]$.
 - Compute the trapdoor $\mathbf{T}_{[\mathbf{A} | \mathbf{B}_l + x_l \mathbf{G}]}$ by running the algorithm (Section 2.6) $\text{ExtendLeft}(\mathbf{A}, (x_l - x_l^*)\mathbf{G}, \mathbf{T}_{\mathbf{G}}, \mathbf{S}_l^*)$, then obtain the trapdoor $\mathbf{T}_{[\mathbf{A} | \mathbf{B}_l + x_l \mathbf{G} | \mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_{l-1} + x_{l-1} \mathbf{G}]}$ by running $\text{ExtendRight}([\mathbf{A} | \mathbf{B}_l + x_l \mathbf{G}], \mathbf{T}_{[\mathbf{A} | \mathbf{B}_l + x_l \mathbf{G}]}, [\mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_{l-1} + x_{l-1} \mathbf{G}])$.
 - Construct the trapdoor $\mathbf{T}_{\mathbf{H}_{\mathbf{x}}}$ by switching the rows of the matrix $\mathbf{T}_{[\mathbf{A} | \mathbf{B}_l + x_l \mathbf{G} | \mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_{l-1} + x_{l-1} \mathbf{G}]}$, which is short (refer Section 2.6).
 - Sample $\mathbf{R}_{\mathbf{x}} \leftarrow \text{SampleD}(\mathbf{H}_{\mathbf{x}}, \mathbf{T}_{\mathbf{H}_{\mathbf{x}}}, \mathbf{U}, \sigma)$ such that $\mathbf{H}_{\mathbf{x}} \cdot \mathbf{R}_{\mathbf{x}} = \mathbf{U}$.
 - Select an attribute vector $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
 - Construct $\mathbf{H}_{\mathbf{y}} = [\mathbf{A} | \mathbf{B}_1 + y_1 \mathbf{G} | \dots | \mathbf{B}_l + y_l \mathbf{G}]$.

- Choose random matrix $\mathbf{R}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times (l+1)md}$, sample error matrices $\mathbf{R}_2 \leftarrow \chi^{(l+1)m \times (l+1)md}$ and $\mathbf{R}_3 \leftarrow \chi^{m \times (l+1)md}$.
- Construct a simulated key from $\mathbf{x}$ to $\mathbf{y}$ as:

$$\text{rek}_{\mathbf{x} \rightarrow \mathbf{y}}^{(\text{sim})} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_1 + y_1 \mathbf{G} | \cdots | \mathbf{B}_l + y_l \mathbf{G}]^\top \mathbf{R}_1 & \mathbf{0}_{(l+1)m \times m} \\ \mathbf{U}^\top \mathbf{R}_1 + \mathbf{R}_3 - \text{P2}(\mathbf{R}_\mathbf{x})^\top & \mathbf{I}_{m \times m} \end{bmatrix}$$

- Compute the 1-hop re-encrypted ciphertext as:

$$\bar{\text{ct}}_{\mathbf{y}}^{(r)} = \text{rek}_{\mathbf{x} \rightarrow \mathbf{y}}^{(\text{sim})} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{\text{out}} \end{bmatrix} \in \mathbb{Z}_q^{(l+2)m}$$

- Output the final ciphertext $\hat{\text{ct}}_{f_2}^{(r)} \leftarrow \text{Eval.AND}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_{\mathbf{y}}^{(r)}, \bar{\text{ct}}_{\mathbf{y}}^{(r)})$.

Theorem 3 (Security). *The AB-FHPRE scheme given in Section 6 is selective IND-HRA secure for classes of function $\mathcal{F}, \mathcal{G}$ under the assumptions of the circular security, the hardness of $\text{DLWE}_{n,q,\chi}$, and the hardness of $\text{RLWE}_{N,Q,\bar{\chi}}$.*

Proof. The security proof proceeds in a sequence of games:

Game 0. This is the original IND-HRA security game from Definition 6.

Game 1. In Game 1, the challenger generates the public parameter as $\text{Setup}_{\text{sim}}$. In Query Phase 1 and Query Phase 2, the challenger will adaptively answer queries issued by the adversary $\mathcal{A}$ as follows:

- $\mathcal{O}^{\text{KeyGen}}$: This is identical to the $\mathcal{O}^{\text{KeyGen}}$ from Definition 6:
- $\mathcal{O}^{\text{ReKeyGen}}$:
 - If $f_1(\mathbf{x}^*) \neq 0$, compute $\text{sk}_{f_1} \leftarrow \mathcal{O}^{\text{KeyGen}}$, then output the $\text{rek}_{f_1 \rightarrow f_2}$ by running ReKeyGen .
 - If $f_1(\mathbf{x}^*) = 0$, run $\text{ReKeyGen}_{\text{sim}}$.
- $\mathcal{O}^{\text{Enc}}$: This is identical to the $\mathcal{O}^{\text{Enc}}$ from Definition 6.
- $\mathcal{O}^{\text{ReEnc}}$: On input two policies, f_1, f_2 , and a key $(\mathbf{x}, j \leq N_{ct})$, let $\mathbf{c}_\mathbf{x}$ be the ciphertext for the key, do as follows:
 - If $\mathbf{x} = \mathbf{x}^*$, output $\perp$ if there is no value in $\mathcal{CT}$ with key $(\mathbf{x}, j)$; Otherwise, run $\text{ReEnc}_{\text{sim-1}}$.
 - If $\mathbf{x} \neq \mathbf{x}^*$, there are two cases: (1) For $f(\mathbf{x}^*) \neq 0$, compute $\text{rek}_{f_1 \rightarrow f_2}$ by running oracle $\mathcal{O}^{\text{ReKeyGen}}$, then compute the re-encrypted ciphertext by $\text{ReEnc}(\text{pp}, \text{rek}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \mathbf{c}_\mathbf{x})$; (2) For $f(\mathbf{x}^*) = 0$, run $\text{ReEnc}_{\text{sim-2}}$.

As shown in the analysis in Theorem 1 according to Lemma 2, we know that both $\mathbf{AS}_i^*$ and $\mathbf{B}_i = \mathbf{AS}_i^* - x_i^* \mathbf{G}$ are statistically close to uniform. Also, similar to the argument in Theorem 1, the distributions, $(\text{RGSW}_z(X^{h \cdot \mathbf{r}_k[i] \cdot B_r^j}), \text{LWE}_s^{q/q}(hz'_i B_{ks}^j))$ and $(\mathbf{Z}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathcal{R}_Q^{2d_g \times 2}, \mathbf{k}_{h,i,j}^{*(k)} \xleftarrow{\$} \mathbb{Z}_q^{n+1})$ are computationally indistinguishable. Moreover, the key ark_f^* works as a valid simulation of an auxiliary recovery key for queries with $f(\mathbf{x}^*) = 0$. Since $\mathbf{AS}_i^*$ is statistically close to uniform, the matrix, $\mathbf{U} = [\mathbf{A}|\mathbf{AS}_1^*| \cdots |\mathbf{AS}_l^*] \cdot \mathbf{R}_{\mathbf{x}^*}$ is also statistical close to uniform. Thus, Game 0 is computationally indistinguishable from Game 1.

Game 2. Game 2 differs from Game 1 in the way by which the challenge ciphertext, ct_β^* , are generated. In Game 2, $\text{ct}_\beta^* = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$ is chosen in $\mathbb{Z}_q^{(l+2)m}$ at uniformly random. Since now the challenge ciphertext is completely independent from β , the advantage of $\mathcal{A}$ is 0. It remains to show Game 2 is indistinguishable from Game 1. We show this by a reduction from the DLWE problem.

Reduction from DLWE. Assuming $\mathcal{A}$ has a non-negligible advantage in distinguishing Game 1 and Game 2, we use $\mathcal{A}$ to construct a DLWE algorithm $\mathcal{B}$.

- **DLWE Instance.** $\mathcal{B}$ begins by obtaining an DLWE challenge instance with two random matrices $\mathbf{A}, \mathbf{U} \in \mathbb{Z}_q^{n \times m}$ and two vector $\mathbf{c}_{in}, \mathbf{c}_{out} \in \mathbb{Z}_q^m$. Here, $\mathbf{c}_{in}$ and $\mathbf{c}_{out}$ are either random in $\mathbb{Z}_q^m$ or as distributions

$$\mathbf{c}_{in} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_0 \quad \text{and} \quad \mathbf{c}_{out} = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_1 \quad (8)$$

where $\mathbf{s} \in \mathbb{Z}_q^n$ and $\mathbf{e}_0, \mathbf{e}_1 \leftarrow \chi^m$. The algorithm $\mathcal{B}$ is intended to distinguish these two cases using $\mathcal{A}$.

- **Initial.** $\mathcal{A}$ sends a target attribute vector $\mathbf{x}^* = (x_1^*, \dots, x_l^*) \in \mathbb{Z}_q^l$ to $\mathcal{B}$.
- **Setup.** $\mathcal{B}$ generate the public parameter pp as in Game 1: Choose l random matrices $\mathbf{S}_1^*, \dots, \mathbf{S}_l^* \xleftarrow{\$} \{1, -1\}^{m \times m}$ and compute $\mathbf{B}_i = \mathbf{A}\mathbf{S}_i^* - x_i^* \mathbf{G}$ for $i \in [l]$. $\mathcal{B}$ then sends $\mathcal{A}$ the public parameters, pp .
- **Query Phase 1.** $\mathcal{B}$ responds to $\mathcal{A}$'s queries as in Game 1.
- **Challenge.** $\mathcal{A}$ sends $\mathcal{B}$ two messages $\mu_0, \mu_1 \in \{0, 1\}^m$. Then $\mathcal{B}$ chooses a random bit $\beta \xleftarrow{\$} \{0, 1\}$, then computes and sends $\mathcal{A}$ the challenge ciphertext $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$, where $\mathbf{c}^* = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{c}_{in} \in \mathbb{Z}_q^{(l+1)m}$, and $\mathbf{c}_{out}^* = \mathbf{c}_{out} + \left\lfloor \frac{q}{4} \right\rfloor \mu_\beta \in \mathbb{Z}_q^m$. If $(\mathbf{c}_{in}, \mathbf{c}_{out})$ is hold as a random distribution, $\mathbf{c}^*$ in the challenge ciphertext is statistically close to uniform based on the standard leftover hash lemma. Since $\mathbf{c}_{out}$ is random, $\mathbf{c}_{out}^*$ is also random. Thus, the challenge ciphertext $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$ is statistically close to uniform, as in Game 2. Note that from the algorithm Enc , an encryption for $\mathbf{x}^*$ uses the matrix: $\mathbf{H}_{\mathbf{x}} = [\mathbf{A} | \mathbf{B}_1 + x_1^* \mathbf{G} | \dots | \mathbf{B}_l + x_l^* \mathbf{G}] = [\mathbf{A} | \mathbf{A}\mathbf{S}_1^* | \dots | \mathbf{A}\mathbf{S}_l^*]$. If $(\mathbf{c}_{in}, \mathbf{c}_{out})$ are pseudorandom distributions as in (8), we have:

$$\mathbf{c}^* = [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot (\mathbf{A}^\top \mathbf{s} + \mathbf{e}_0) = \mathbf{H}_{\mathbf{x}}^\top \mathbf{s} + [\mathbf{I}_m | \mathbf{S}_1^* | \dots | \mathbf{S}_l^*]^\top \cdot \mathbf{e}_0$$

and $\mathbf{c}_{out}^* = \mathbf{c}_{out} + \left\lfloor \frac{q}{4} \right\rfloor \mu_\beta = \mathbf{U}^\top \mathbf{s} + \mathbf{e}_1 + \left\lfloor \frac{q}{4} \right\rfloor \mu_\beta$. It shows that $\text{ct}_\beta^* = (\mathbf{c}^*, \mathbf{c}_{out}^*)$ is a valid encryption of μ_β , with the same distribution as in Game 1.

- **Query Phase 2.** $\mathcal{B}$ responds to $\mathcal{A}$'s queries as in Query Phase 1.
- **Guess.** $\mathcal{A}$ guesses if it is interacting with a Game 1 or Game 2 challenger. $\mathcal{B}$ outputs its answer to the DLWE challenge based on $\mathcal{A}$'s output.

Based on the analysis above, when the LWE challenge is a random instance, $\mathcal{A}$'s view is as in Game 2. When the LWE challenge is a pseudorandom instance as in (8), $\mathcal{A}$'s view is as in Game 1. Therefore, $\mathcal{B}$'s advantage in solving the DLWE problem is at least as large as $\mathcal{A}$'s advantage in distinguishing Game 1 and Game 2, which completes the proof.